Mechatronics system design

23EARC302

Project report on

Two Wheeled Self-Balancing Bot

submitted to the Department of Automation & Robotics of the

KLE Technological University in partial fulfillment

of the requirements for the degree of

Bachelor of Engineering

in

Automation & Robotics

Prashant Hegde	01FE23BAR001
Rohith Meti	01FE23BAR007
Akash Singh	01FE23BAR025
Ilaf	01FE23BAR028
Himanshu Pal	01FE23BAR029

Submitted by

Under the guidance of

Mrs. Shamshad Shirgeri and Mrs. Shilpa Tanvashi and Ms. Girija

Dept of Automation & Robotics

2025-2026

KLE TECHNOLOGICAL UNIVERSITY

DEPARTMENT OF AUTOMATION & ROBOTICS

Certificate

This is to certify that Mechatronics system design - Project report entitled Two Wheeled Self Balancing Robot made smart and simple is a Bonafide work carried out by Prashant Hegde, Rohith, Himanshu Pal, Akash Singh and Ilaf in partial fulfillment for the award of degree of Bachelor of Engineering in Automation & Robotics of KLE Technological University, Hubballi during the year 2025-2026.

Guide

Designation

External Viva

Name of the Examiner Signature with Date

1.
2.

ACKNOWLEDGEMENT

We would like to extend our heartfelt gratitude and appreciation to everyone who helped bring this project to a successful conclusion.

First and foremost, we want to express our sincere thanks to Dr. Vinayak N. Kulkarni, who oversaw our research and gave us tremendous advice and assistance. Their opinions and suggestions were really helpful in helping us to refine and improve our work.

We would provide our heartly thanks to the faculty in-charge, Mrs. Shamshad Shirgeri and Mrs. Shilpa Tanvashi and Ms. Girija for their continuous involvement, assigning challenging tasks and quite an ease of delivery of content. The examples taken up in-session were easy to understand and practice, demonstrated all the tools required for the project fulfillment.

We also like to express our gratitude to the teachers and employees of Automation and Robotics, KLE Technological University in Hubballi for their support and use of the project's resources. The development of the initiative was greatly aided by their dedication to supporting new ventures and to academic achievement.

01FE23BAR001 01FE 23BAR007 01FE23BAR025

01FE23BAR028 01FE23BAR029

ABSTRACT

The design and control of a two-wheeled self-balancing robot present a classic engineering challenge involving non-linear dynamics and inherent system instability. This project focuses on the development of a custom-designed mechatronic system capable of maintaining an upright equilibrium while performing cargo transportation maneuvers. The primary objective was to implement a robust control strategy that decouples the high-level logic from the physical simulation, allowing for rapid prototyping and validation of control algorithms.

The central control architecture, effectively functioning as the "Brain" of the system, was developed within the MATLAB and Simulink environment. This computational layer is responsible for all decision-making processes, including data acquisition, signal processing, and stability management. A custom-built Simulink block handles the real-time decoding of sensor data, converting raw Quaternion orientation streams into readable Euler angles. The core stabilization is achieved through a tuned Proportional-Integral-Derivative (PID) controller that dynamically calculates the necessary torque requirements to counteract gravitational forces and external disturbances. The MATLAB architecture was rigorously configured with a fixed-step solver to minimize latency and ensure precise synchronization with the simulation clock.

To validate this control logic against real-world scenarios, the physical modeling and simulation were executed in Ignition Gazebo. This environment serves as the "Body" of the robot, simulating complex physical interactions such as wheel friction, chassis inertia, collision dynamics, and gravitational acceleration. The communication between the MATLAB controller and the Gazebo physics engine is mediated by ROS 2 (Robot Operating System), which establishes a bidirectional bridge for transmitting IMU sensor data and receiving motor velocity commands.

The project also addresses significant integration challenges, specifically the resolution of URDF compatibility issues through manual refactoring of Xacro files to ensure accurate physical modeling. The final result is a stable, co-simulated robotic platform that successfully demonstrates self-balancing capabilities alongside user- controlled maneuverability (forward, backward, and turning), confirming the viability of the hybrid MATLAB- ROS 2 architecture for complex mechatronic system design.

Keywords: Self-Balancing Robot, MATLAB Simulink, Ignition Gazebo, ROS 2, PID Control, Inverted Pendulum, Co-Simulation

TABLE OF CONTENTS

Chapter No.	Title of the Chapter	Page No.
—	Certificate	i
—	Declaration	ii
—	Acknowledgement	iii
—	Abstract	iv
—	Table of Contents	v
—	List of Figures	vi-vii
—	List of Tables	viii
Chapter 1	Introduction	1–3
Chapter 2	Literature Review	4
Chapter 3	Mechatronics System Design Methodology	5-12
Chapter 4	Simulation Environment Integration	13-25
Chapter 5	Software and Control System Design	26-49
Chapter 6	Testing and Validation	50-55
Chapter 7	Results and Discussion	56-63
Chapter 8	Applications, Limitations, and Future Scope	64-66
Chapter 9	Conclusion	67
—	References	68

List of Figures

Figure Number	Title of the figures	Page Number
1	V-Model	5
2	Conceptual Design	6
3	3d- model	7
4	STL files	14
5	Visualizing CoM and Joints in rvs2	19
6	Visualizing CoM in rvs2	19
7	Spawned bot in Ignition Gazebo	23
8	Visualizing CoM and joints and axis in Ignition Gazebo	23
9	Flowchart	26
10	System Design Linked to Requirements In MATLAB	28
11	System architecture	28
12	IMU Subsystem	29
13	IMU graph Data visualization tilt	32
14	Bot leaning over due to gravity when PID OFF	32
15	User Interface	34
16	User interface in terminal	36
17	User interface graph	37
18	PID subsystem	37
19	Imu Data after turning on PID	42
20	PWM Data	43
21	PWM signal	43
22	Motor Driver Subsystem	44
23	Motor driver converting PWM signal into 12 v	45
24	Motors Actuator Subsystem	46
25	Balancing voltage is converted to balancing velocity	48
26	RAW IMU data from Ignition Gazebo	50
27	IMU data same as raw data published by Ignition Gazebo	51
28	Motor Actuation	53
29	PWM signal	54
30	Bot with Payload	55
31	Difference in imu data before and after Pid	56
32	Bot collapse on to ground without PID	57
33	Bot self-balancing after PID turned ON	58

34	Before and After input from user command	59
35	Bot self-balancing with payload placed on it	61
36	Bot climbing on ramp	63

List of Tables

Figure Number	Title of the table	Page Number
1	System Mass Budget	10
2	Kinematic Joint Definitions	12
3	Requirements Collection in MATLAB	27

CHAPTER 1

INTRODUCTION

1.1 Background: The Inverted Pendulum Problem

The two-wheeled self-balancing robot is a practical implementation of the classical Inverted Pendulum problem. In standard mechanics, a pendulum suspended from a pivot point is naturally stable because its center of mass is located below the pivot; gravity acts to restore it to the equilibrium position.

Conversely, an inverted pendulum has its center of mass located above its pivot point. This configuration is inherently unstable and non-linear. Even a microscopic displacement from the vertical axis results in a gravitational torque that accelerates the pendulum further away from equilibrium, causing it to fall.

To maintain an upright position, the system requires continuous active control. The robot must drive its wheels in the direction of the fall to generate a reaction force that pushes the center of mass back over the pivot point. This dynamic stabilization is the fundamental principle behind modern transportation devices like Segways, self-balancing wheelchairs, and even rocket guidance systems during launch.

1.2 Motivation

The motivation for this project stems from two primary factors: the need for agile logistics solutions and the importance of safe control system validation.

Agile Cargo Transportation: Traditional four-wheeled mobile robots (AGVs) require a wide turning radius and significant floor space. A two-wheeled self-balancing robot offers a zero turning radius and a smaller footprint, making it ideal for navigating narrow aisles in warehouses or crowded office environments.

Safe Prototyping (Hardware-in-the-Loop): Developing control algorithms for unstable systems on physical hardware is risky and expensive; a single coding error can result in catastrophic damage to the robot's structure or electronics. By decoupling the control logic (Brain) from the physical robot (Body) using a high-fidelity simulation, we can push the system to its limits without physical consequences. This project aims to bridge the gap between theoretical control design in MATLAB and realistic physics simulation in Gazebo.

1.3 Problem Statement

While self-balancing robots are a common academic subject, developing a customized platform for cargo transportation presents specific engineering challenges that this project aims to address:

Mechanical Instability with Payload: Unlike lightweight hobbyist robots, a cargo-carrying robot has a variable center of mass. Adding a payload shifts the system's inertia, making standard stabilization techniques insufficient without robust tuning.

Simulation Interoperability Gap: Modern engineering often relies on disjointed tools. Designing

in Autodesk Fusion 360, simulating in Ignition Gazebo, and controlling via MATLAB presents a significant integration challenge. A major problem addressed in this project is the failure of standard ROS 2 exporters to generate valid physical models, requiring a manual re-engineering of the robot's description files (URDF/Xacro) to ensure accurate collision and friction properties.

Real-Time Synchronization: Establishing a feedback loop between an external controller (MATLAB) and a physics engine (Gazebo) requires precise time synchronization to avoid latency-induced instability, often referred to as the "Staircase Effect" in signal processing.

1.4 Objectives

The primary objectives of this project are defined as follows:

Mechanical Design: To design a custom robot chassis in Autodesk Fusion 360 that is structurally capable of holding a cargo payload while maintaining a center of mass distribution suitable for inverted pendulum dynamics.

Simulation Environment Setup: To successfully import the mechanical model into the Ignition Gazebo environment via ROS 2, ensuring that all physical properties (mass, inertia, wheel friction) are accurately represented.

Control System Architecture: To develop a modular control architecture where MATLAB/Simulink functions as the central processing unit ("Brain") to process sensor data and calculate control signals.

Stabilization and Maneuverability: To implement and tune a Proportional-Integral-Derivative (PID) controller that stabilizes the robot and allows for user-controlled maneuvering (forward, backward, and turning) using the WASD teleoperation protocol.

1.5 Scope of the Project

The scope of this work is focused on the design, simulation, and control validation of the robotic system.

3D Modeling and Rigid Body Dynamics analysis.

Co-Simulation using ROS 2 Humble, Ignition Gazebo, and MATLAB R2023b+.

Implementation of IMU sensor fusion (Quaternion to Euler decoding).

Implementation of PID control logic for active balancing.

Teleoperation logic for maneuvering on flat surfaces.

Out of Scope:

Physical fabrication and hardware assembly of the robot (reserved for future phases).

Autonomous navigation using SLAM (Simultaneous Localization and Mapping).

Adaptive control for extreme terrain or highly variable dynamic loads (beyond basic cargo weights).

CHAPTER 2
Literature Review

1 Paper Name: A Study of Two-Wheel Self-Balancing Robot Dynamics for Control Systems Learning
Author: R. A. D. D. Dharmasiri and M. K. Jayananda
Published: July 2024
Website: Vidyodaya Journal of Science (journals.sjp.ac.lk)

2 Paper Name: A Two Wheel Self-Balancing Vehicle
Author: Rastislav Ďuriš, Eva Labašová, Jarmila Oravcová, and Štefan Šimon
Published: September 2021
Website: Sciendo / Research Papers Faculty of Materials Science and Technology (sciendo.com)

3 Paper Name: Mathematical Modelling of Hover Board Author: Dilip I. Sangotra, Mohan Mendhe, Surendra D. Kshirsagar, and Ram Tamboli Published: May 2021
Website: Easy Chair / ResearchGate (easychair.org)

4 Paper Name: Modelling, Simulation, and Optimal Control for Two-Wheeled Self-Balancing Robot
Author: Modestus Oliver Asali, Ferry Hadary, and Bomo Wibowo Sanjaya
Published: August 2017
Website: International Journal of Electrical and Computer Engineering (iaescore.com)

5 Paper Name: Two-wheel Balancing Robot; Review on Control Methods and Experiments
Author: M.R.M. Romlay, M.I. Azhar, S.F. Toha, and M.M. Rashid
Published: March 2019
Website: International Journal of Recent Technology and Engineering (ijrte.org)

CHAPTER 3

Mechatronics System Design Methodology

3.1 Methodology:

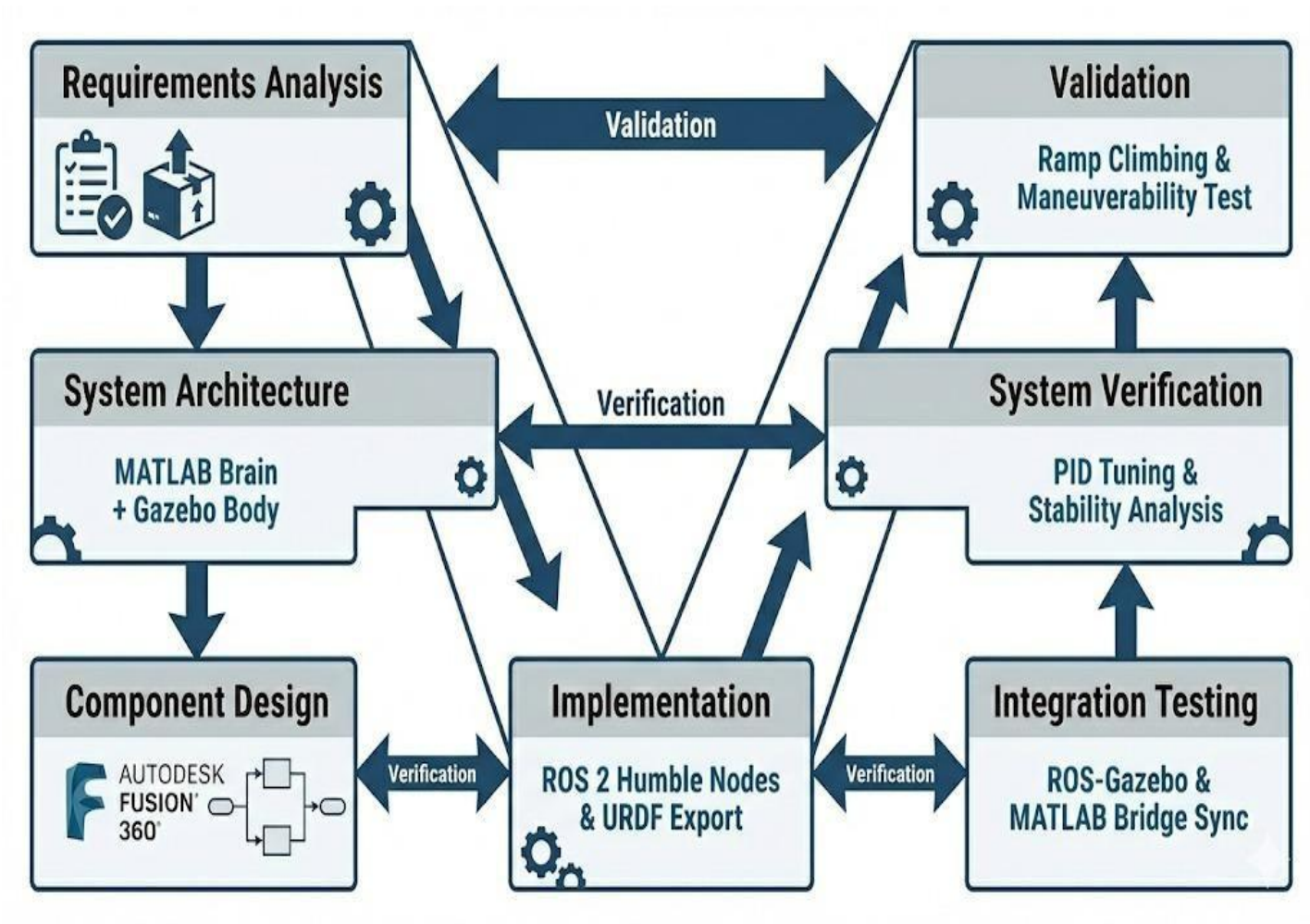


Figure1: V-Model

3.2 Conceptual Design

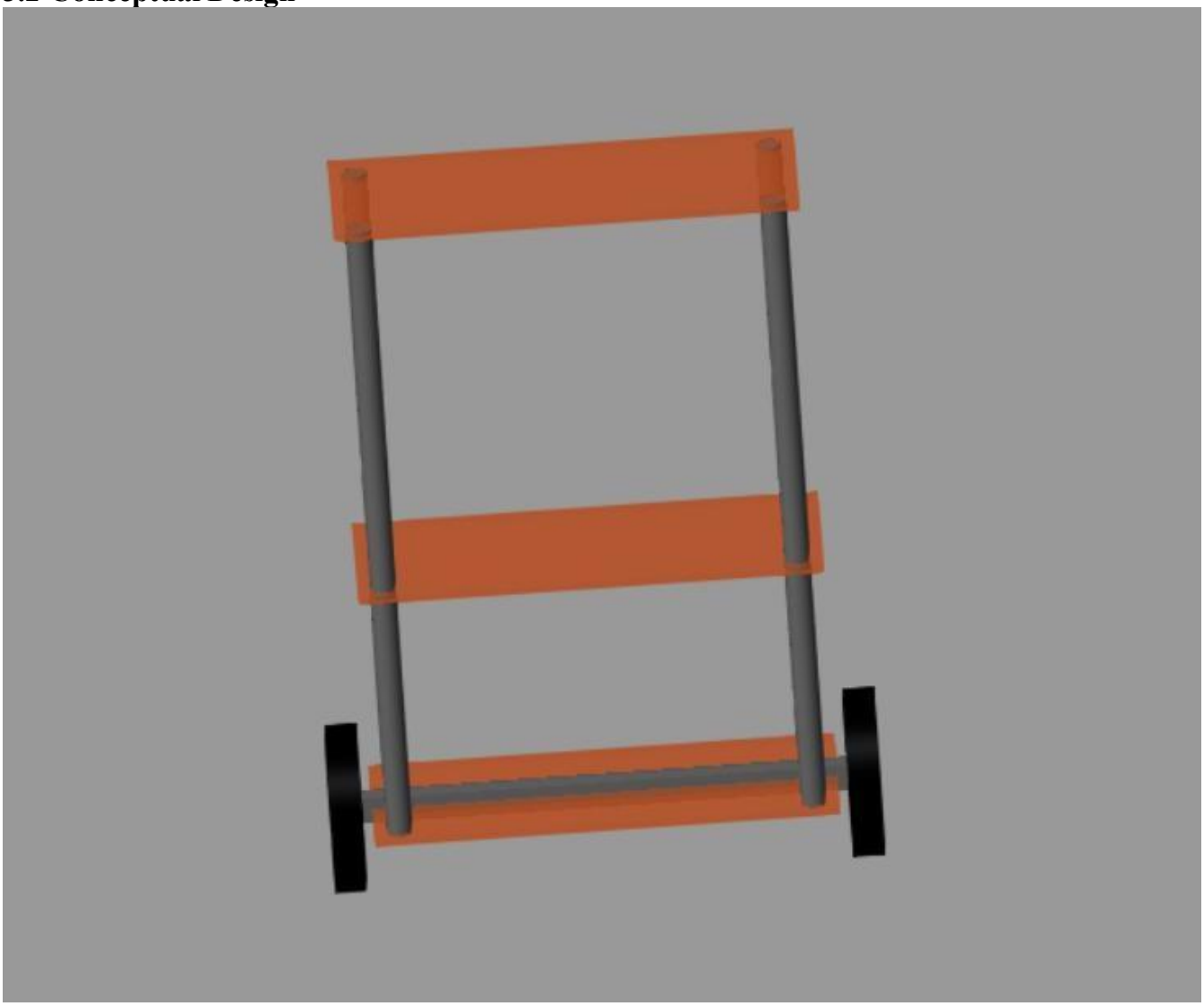


Figure 2: Conceptual Design

The initial design phase focused on developing a robust mobile platform capable of performing practical logistics tasks, rather than serving as a simple educational demonstrator. The conceptual sketch (Figure 3.1) illustrates the fundamental layout of the system: a differential-drive inverted pendulum robot equipped with a dedicated upper cargo deck.

3.2 Autodesk fusion

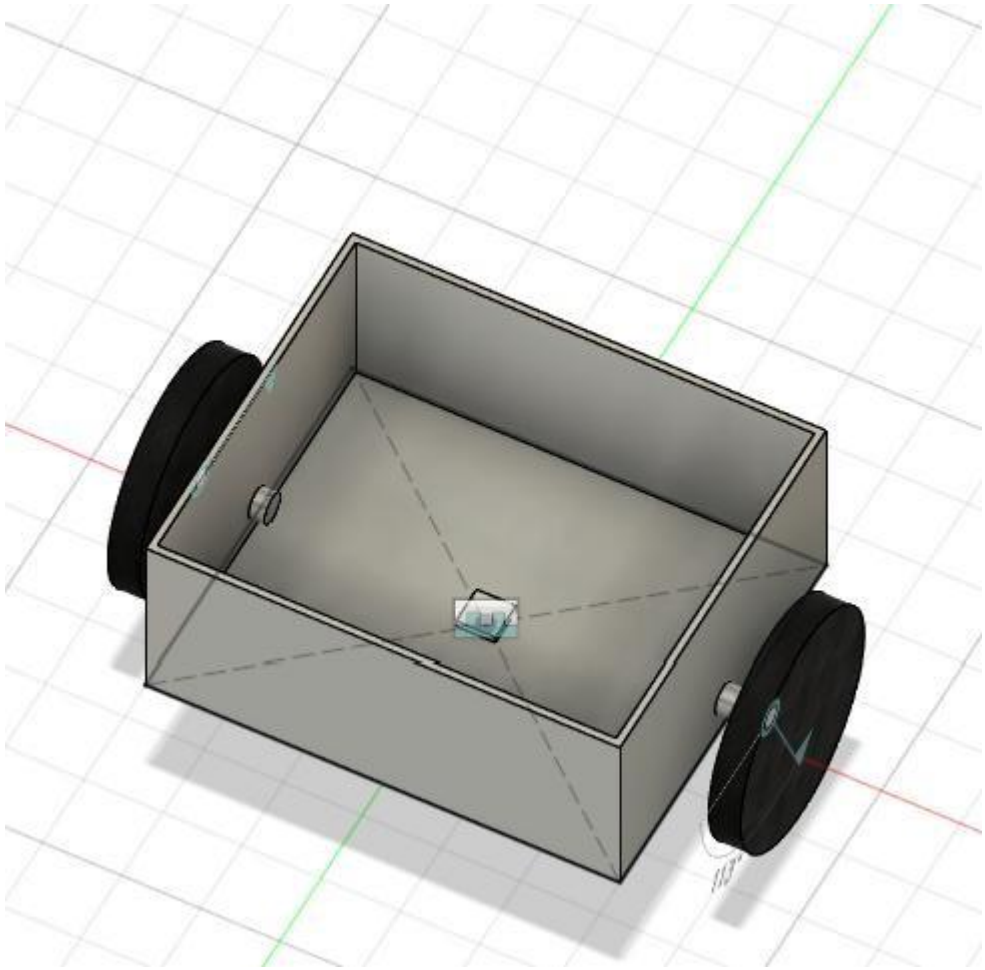


Figure 3: 3d- model

3.3 Mechanical Modeling in Autodesk Fusion 360

The conceptual sketch was translated into a high-fidelity three-dimensional model using Autodesk Fusion 360. The complete assembly consists of the following four primary rigid bodies:

1. Base Link (Chassis) – The main structural body of the robot
2. Right Wheel – Actuator for the right side
3. Left Wheel – Actuator for the left side
4. IMU Link – Virtual or physical mounting point for the inertial sensor

This modeling approach ensures accurate mass distribution and inertial properties for subsequent dynamic simulation in Gazebo

3.3.1 Base Link (Chassis) Analysis

The base_link serves as the primary structural component of the robot. Unlike typical hobby- grade robots constructed from acrylic sheets or lightweight 3D-printed plastics, this chassis was modeled using industrial-grade specifications to simulate a realistic cargo-carrying scenario.

Physical Properties

Material Selection

The chassis material was defined as Aluminum 6061, selected for its balance of strength, rigidity, and realistic density:

$\rho = 2700 \text{ kg/m}^3$

This choice ensures that the simulation accurately reflects the momentum and inertial effects of a metallic structure.

Geometric Dimensions

A bounding-box analysis confirms the substantial footprint of the robot, contributing to enhanced stability:

- Length: 0.72 m
- Width: 0.50 m
- Height: 0.25 m

Mass Budget

The total mass of the chassis alone is:

$m = 22.64 \text{ kg}$

This deliberately high mass is intended to stress-test the balancing controller. While a heavier robot exhibits slower falling dynamics due to increased rotational inertia, it demands significantly higher motor torque to recover from disturbances.

Inertial Properties

Accurate inertial parameters are critical for the Gazebo physics engine, which computes angular acceleration based on:

$\tau = I \cdot \alpha$

Using the Fusion 360 mass properties tool, the following moments of inertia were obtained at the center of mass:

- $I_{xx} = 0.903 \text{ kg\cdotpm}^2$
  (Resistance to rolling sideways)
- $I_{yy} = 1.221 \text{ kg\cdotpm}^2$
  (Resistance to forward/backward pitching – critical for balancing)
- $I_{zz} = 1.828 \text{ kg\cdotpm}^2$
  (Resistance to yaw/spinning motion)

These high inertia values confirm that the robot possesses strong resistance to rapid external disturbances, provided the PID control loop is appropriately tuned to manage the 22 kg static load.

3.3.2 Wheel Analysis

Physical Properties
Material Selection
The wheels are modeled using Nitrile Rubber, a material selected for its high friction characteristics. In simulation, this corresponds to a coefficient of friction of approximately:
$\mu \approx 0.9$

A high friction coefficient is essential to prevent wheel slippage during aggressive acceleration and rapid corrective maneuvers, which are common in self-balancing systems.

Geometric Dimensions
Bounding box analysis of the wheel model indicates the following dimensions:

- Wheel Diameter: 0.30 m
- Wheel Radius:

$r = 0.15 \text{ m}$

- Tire Width: 0.04 m

The relatively large wheel diameter improves the robot's ability to roll over small surface irregularities and obstacles without inducing sudden disturbances to the inverted pendulum body.

Mass Budget
Each wheel has a mass of:
$m = 3.393 \text{ kg}$

The combined mass of both wheels is approximately:
$m_{\text{total}} \approx 6.8 \text{ kg}$

This additional mass is concentrated near the ground plane, which effectively lowers the overall center of gravity of the system and contributes to enhanced passive stability during balancing.

Inertial Properties
The rotational inertia of the wheels directly affects how quickly the motors can accelerate or decelerate them. Fusion 360 mass property analysis at the center of mass yielded the following inertia values:

- Axis of Rotation:

$I_{xx} = 0.038 \text{ kg\cdotpm}^2$

This value closely matches the theoretical inertia of a solid cylinder:

$$I = \frac{1}{2} m r^2$$

confirming the accuracy and physical realism of the wheel model.

- Orthogonal Axes:

$I_{yy} = 0.02 \text{ kg\cdotpm}^2$
$I_{zz} = 0.02 \text{ kg\cdotpm}^2$

Design Implications for Balancing Control
The relatively low rotational inertia of the wheels
$I_{\text{wheel}} = 0.038\ \text{kg\cdotpm}^2$

compared to the chassis pitch inertia
$I_{\text{chassis}} = 1.221\ \text{kg\cdotpm}^2$

is highly favorable for balancing control. This ensures that the motors expend the majority of their energy correcting the chassis tilt, rather than overcoming the wheels' own resistance to rotation. As a result, the system achieves faster corrective response and improved overall stability.

3.4 Physical Constraints and Mass Budget

For an inverted pendulum system, mass distribution is the defining factor governing controllability. Unlike statically stable mobile robots—where lowering the center of gravity always improves stability—an inverted pendulum requires a deliberate balance between the Centre of Mass (CoM) location and the available motor torque.
The robot must be sufficiently unstable to enable active balancing, while still remaining within the recoverable torque limits of the drive system.

3.4.1 Mass Budget Summary
Based on the material properties assigned in Autodesk Fusion 360, the total system mass was computed to verify that it remains within the feasible operating range of standard high-torque DC motors commonly used in mobile robotics.

Component	Material	Mass per Unit (kg)	Quantity	Total Mass (kg)
Chassis (Base Link)	Aluminum 6061	22.64	1	22.64
Drive Wheels	Rubber (Nitrile)	3.393	2	6.786
IMU Module	ABS Plastic	0.026	1	0.026
Total System Mass	—	—	—	29.452 kg

Table 1: System Mass Budget

3.4.2 Constraint Analysis
Heavy-Duty Classification
The total system mass of approximately:
$m \approx 29.5\ \text{kg}$

places this robot firmly in the heavy-duty category when compared to typical educational self-balancing platforms, which usually weigh less than 2 kg. This design choice intentionally pushes the limits of controller performance and actuator capability.

The Inertia Advantage

The large chassis mass ($22.6\ \mathrm{kg}$) results in a high pitch-axis moment of inertia:

$I_{yy} = 1.22\ \mathrm{kg\backslash cdotpm^2}$

From a physical standpoint, this means the robot falls more slowly when disturbed. Since angular acceleration is inversely proportional to inertia:

$$\alpha = \frac{\tau}{I}$$

a larger inertia reduces angular acceleration caused by gravity, giving the control loop additional milliseconds to detect and correct balance errors. This characteristic significantly improves controller observability and tuning robustness.

The Torque Constraint

The primary trade-off of increased mass is the torque requirement. Recovering a $29.5\ \mathrm{kg}$system from a tilt demands substantial motor torque. If the robot exceeds a critical tilt angle (approximately 15°), the motors may reach their torque saturation limit, failing to generate sufficient corrective force to return the CoM over the wheel axle.

This physical constraint directly influenced the safety limits and saturation handling implemented later in the PID controller.

3.5 Centre of Mass (CoM) Considerations

The location of the Centre of Mass is fundamental to the inverted pendulum dynamics.

Vertical CoM (Z-axis)

The simulation assumes the CoM is located significantly above the wheel axle axis, creating an inherently unstable configuration. This instability stores gravitational potential energy, which the control system actively manages to maintain balance.

Longitudinal CoM (X-axis)

In the nominal CAD model, the CoM is perfectly cantered:

$X = 0$

However, in real cargo-handling applications, payload placement may cause forward or backward CoM shifts. The controller design therefore assumes variable longitudinal CoM positions, ensuring robustness against payload-induced imbalance.

3.6 Joint Definitions (Continuous vs. Fixed)

To convert the static Fusion 360 assembly into a dynamic kinematic chain compatible with ROS 2 and Gazebo, the rigid-body relationships were defined using the URDF (Unified Robot Description Format).

Joint Name	Parent Link	Child Link	Joint Type	DoF	Description
left_wheel_joint	base_link	wheel_left_link	Continuous	1 (Rotation about Y)	Allows infinite wheel rotation; actuated via Gazebo diff-drive plugin
right_wheel_joint	base_link	wheel_right_link	Continuous	1 (Rotation about Y)	Identical to left joint; enables differential steering
imu_joint	base_link	imu_link	Fixed	0	Rigidly locks the IMU to the chassis for accurate orientation sensing

Table 2: Kinematic Joint Definitions

CHAPTER 4

Simulation Environment Integration

4.1 The ROS 2 URDF Exporter Challenge

A critical phase in Mechatronics System Design is the Sim-to-Real workflow—transferring the high-fidelity mechanical model from a CAD environment (Autodesk Fusion 360) into a physics-based simulation environment (Ignition Gazebo). The standard file format for this description is the URDF (Unified Robot Description Format). However, Autodesk Fusion 360 does not natively support URDF export, necessitating the use of third-party scripts.

This project encountered significant interoperability issues when attempting to use standard Fusion to ROS 2 export tools, leading to the development of a custom hybrid workflow to ensure simulation accuracy.

4.1.1 Issues with Native Exporters

Initial attempts were made to utilize community-developed ROS 2 exporters to generate the robot description directly. These attempts failed to produce a simulation-ready model due to three specific technical deficiencies:

1. Broken Asset Paths
   The exporters generated absolute system paths (e.g., C:/Users/Admin/Desktop/Project/mesh.stl) rather than the required relative ROS package paths (e.g., package://ros2_twsbr2/meshes/mesh.stl). This made the generated package non-portable, causing immediate build failures when moved to the Linux-based simulation environment.
2. Incorrect Joint Definitions
   The export scripts defaulted to defining the wheel joints as Fixed or Revolute with Limits rather than Continuous joints. For a self-balancing robot, the wheels must rotate infinitely; a joint limit causes the physics engine to abruptly stop the wheel after one rotation, destabilizing the pendulum.
3. Missing Gazebo-Specific Tags
   Standard URDF exporters focus on kinematics (geometry) but ignore dynamics (physics). The generated files lacked critical Gazebo tags required by Ignition Gazebo to define friction coefficients and collision properties. Without these parameters, the robot slid across the simulation surface without generating sufficient traction.

4.1.2 Manual Refactoring of Xacro/URDF Files (The Hybrid Workflow)

To resolve these export failures, a robust Hybrid Workflow was implemented. This involved extracting the raw geometric assets using a stable ROS 1 tool and then manually re-engineering the description logic for ROS 2.

Step 1: Raw Asset Extraction (ROS 1 Standard)

The model was first exported using a stable, legacy ROS 1 plugin. While this tool could not generate a valid ROS 2 package, it successfully produced high-quality STL (Stereolithography) mesh files for the chassis and wheels, preserving the precise dimensions derived from the CAD model.

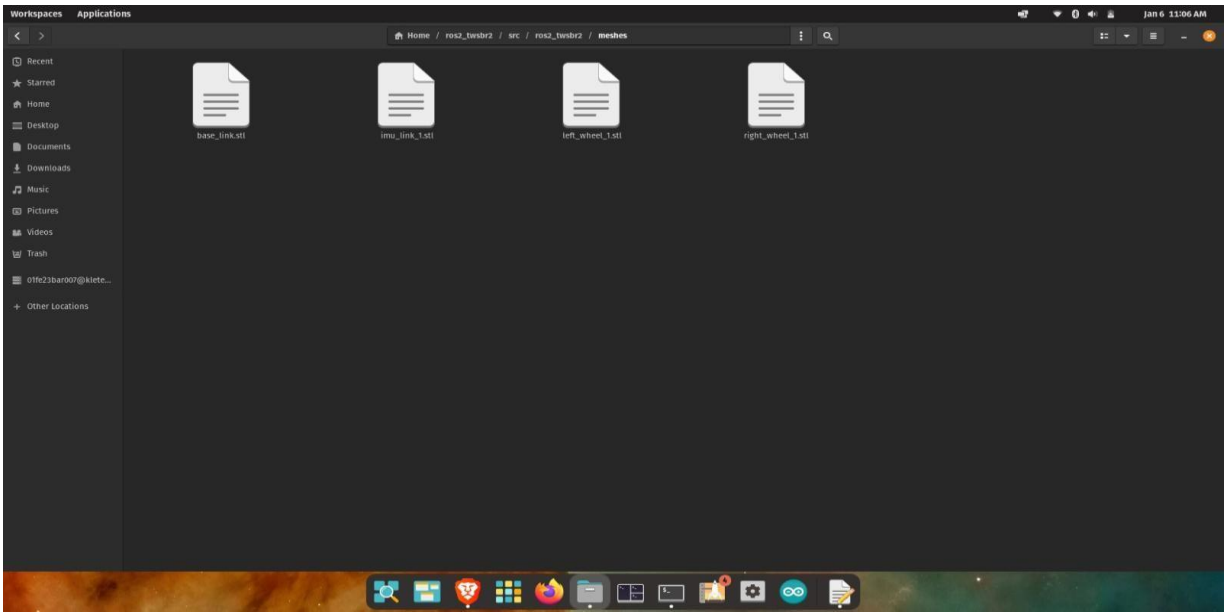


Figure 4: STL files

Step 2: Conversion to Xacro Architecture

The static URDF file was discarded in favor of manually rewriting the robot description in Xacro (XML Macro) format. This conversion provided modularity that standard URDF lacks:

- Parametric Definition

Physical properties such as chassis mass, wheel radius, and wheel separation were defined as variables (Xacro properties) at the top of the file. This allowed for rapid tuning of physical parameters without searching through hundreds of lines of code.

- Inertia Calculation

Instead of hard-coding inertia matrices, Xacro macros were written to calculate inertia tensors dynamically based on the input mass, ensuring that the physics remained consistent even if the mass budget changed.

Step 3: Path and Plugin Injection

The final refactoring step involved manually correcting the mesh file paths to match the ament build system structure (package://ros2_twsbr2/...) and injecting the specific Ignition Gazebo plugins required for sensors and actuators, as detailed in Section 4.3.

4.2 : Simulation Environment Setup (Ignition Gazebo)

4.2.1 Selection of Simulation Engine: Ignition vs. Classic

For the simulation phase of this project, a strategic decision was made to utilize Ignition Gazebo (Fortress) rather than the older and more widely documented Gazebo Classic (version 11). While Gazebo Classic remains the default choice for many undergraduate-level projects, it relies on legacy physics engines and is approaching end-of-life support.

Ignition Gazebo was selected to ensure that the project aligns with modern robotics simulation standards and remains relevant to current and future research practices. This decision introduced a significant technical challenge, as Ignition employs a fundamentally different architecture based on the Entity–Component–System (ECS) paradigm, along with a redesigned plugin and system interface. Unlike Gazebo Classic, Ignition does not rely on monolithic model plugins, requiring a deeper understanding of its modular system-based API.

Adopting this newer simulation platform necessitated manual adaptation of standard ROS 2 drivers and a thorough study of the ignition-gazebo API. This approach demonstrates a deliberate commitment to advanced, future-proof engineering practices rather than reliance on deprecated tooling.

4.2.2 Environment Configuration and Asset Loading

The simulation environment required precise configuration of the ROS 2 launch system to ensure correct loading of the robot model and associated assets within Ignition Gazebo.

Mesh Path Resolution
As identified in the URDF exporter challenge, the chassis and wheel mesh files (STL format) were installed within the ROS 2 package's install/share directory. To allow Ignition Gazebo to locate and render these assets correctly, the IGN_GAZEBO_RESOURCE_PATH environment variable was dynamically injected during the launch sequence. This ensured compatibility with the ament build system while maintaining portability across different machines.

World Generation
A custom simulation world file, ramps.sdf, was developed to evaluate the robot's performance under varying operating conditions. The world includes a standard flat ground plane for controller tuning and stability analysis, as well as inclined planes to test maneuverability and balance control under increased gravitational and traction demands.

4.2.3 Adaptation of URDF for Ignition Physics

Since standard URDF files are primarily designed for Gazebo Classic, the robot description required extensive modification to function correctly within the Ignition physics engine. This was achieved by injecting Ignition-specific XML systems and plugins directly into the Xacro-based robot description.

Key Modifications

1. Physics Material Properties
   The default friction definitions were replaced with Ignition-specific collision and contact parameters. These modifications ensured realistic interaction between the rubber wheels and the ground plane, preventing excessive slipping or penetration during dynamic simulations.
2. Sensor Injection
   The inertial measurement unit (IMU), which existed only as a static component in the CAD model, was converted into an active sensor within the simulation. This was achieved by attaching the libignition-gazebo-imu-system.so plugin to the corresponding link. The sensor was configured with Gaussian noise parameters to emulate the behaviour and imperfections of a real-world MPU6050 IMU.
3. Actuator Interface
   The wheel joints were interfaced with the libignition-gazebo-diff-drive-system.so plugin. This system intercepts velocity commands published on the cmd_vel topic and translates them into joint-level forces and torques. In effect, the plugin functions as a virtual motor driver, enabling closed-loop control of the self-balancing robot within the Ignition simulation environment.

4.3 : Plugin Configuration and Validation

Before implementing dynamic behaviour within the physics engine, it was necessary to validate the robot's kinematic structure and geometric correctness. While the URDF and Xacro files define the robot's physical layout, it is the Ignition Gazebo plugins that enable sensing, actuation, and interaction with the simulated environment. A staged validation approach was therefore adopted, beginning with visualization-based verification and progressing toward full physics simulation.

4.3.1 Kinematic Verification in RViz

Prior to launching the physics-based simulation in Ignition Gazebo, the robot model underwent a static verification process using the RViz2 visualization tool. A dedicated launch file, rviz_only.launch.py, was created to load the robot description without invoking any physics or simulation plugins.

Objectives of RViz Testing

1. Mesh Integrity
   All STL mesh assets corresponding to the chassis and wheels were verified for correct loading, scale, and orientation. This ensured that no unit conversion or coordinate-frame errors were introduced during the CAD-to-URDF workflow.
2. Joint Articulation
   The joint_state_publisher_gui was used to manually actuate the wheel joints. This confirmed that the joints were correctly defined as continuous joints and that rotation occurred about the intended axis (Y-axis). Additionally, the TF tree was inspected to verify logical and consistent frame connectivity between the base, wheels, and sensor links.
3. Collision Geometry Verification
   By enabling collision visualization within RViz, the simplified collision geometries were inspected. The cylindrical collision models were confirmed to align precisely with the visual meshes. This step was critical to ensure accurate contact dynamics and stable behavior once the robot was introduced into the physics engine.

This RViz-based validation step ensured that the robot's kinematic and geometric descriptions were correct before introducing dynamic simulation complexity.

4.3.2 Ignition Gazebo IMU System

The primary sensor used for balancing control in the robot is the inertial measurement unit (IMU). In the physical implementation, this role is fulfilled by an MPU6050 sensor. To replicate this behavior in simulation, the libignition-gazebo-imu- system.so plugin was employed.

Implementation Details

The IMU plugin was injected directly into the Xacro file and attached to the imu_link of the robot model.

Data Output
The plugin publishes sensor_msgs/Imu messages, providing orientation data in quaternion form along with angular velocity measurements. These outputs form the primary feedback inputs for the balancing controller.

Update Rate
The sensor update rate was configured to operate at 100 Hz. This high sampling frequency is essential for stable closed-loop control. Lower update rates introduce latency into the PID control loop, which can cause excessive oscillations or complete loss of balance in an inverted pendulum system.

Noise Modelling
Gaussian noise was intentionally added to the simulated IMU data. This design

choice ensures that the control algorithm is robust to sensor imperfections and better reflects real-world operating conditions rather than relying on idealized, noise-free measurements.

4.3.3 Differential Drive System

To actuate the robot's wheels, the libignition-gazebo-diff-drive-system.so plugin was used as a virtual motor driver. This plugin abstracts the low-level physics of wheel torque application, allowing the control system to operate using high-level velocity commands.

Key Configuration Parameters

Joint Linking
The differential drive plugin was explicitly mapped to the left_wheel_joint and right_wheel_joint defined in the robot description. This ensured that velocity commands were applied correctly to the physical joints responsible for motion.

Input Interface
The plugin subscribes to the standard cmd_vel topic, which carries geometry_msgs/Twist messages. This interface allows the external controller, implemented in MATLAB, to command linear and angular velocities without directly managing individual wheel dynamics.

Odometry Calculation
As a secondary function, the plugin computes wheel-encoder-based odometry and publishes the robot's estimated position and orientation on the Odom topic. Although this information is not directly used for balancing control, it is essential for higher-level maneuvering, navigation, and trajectory planning tasks.

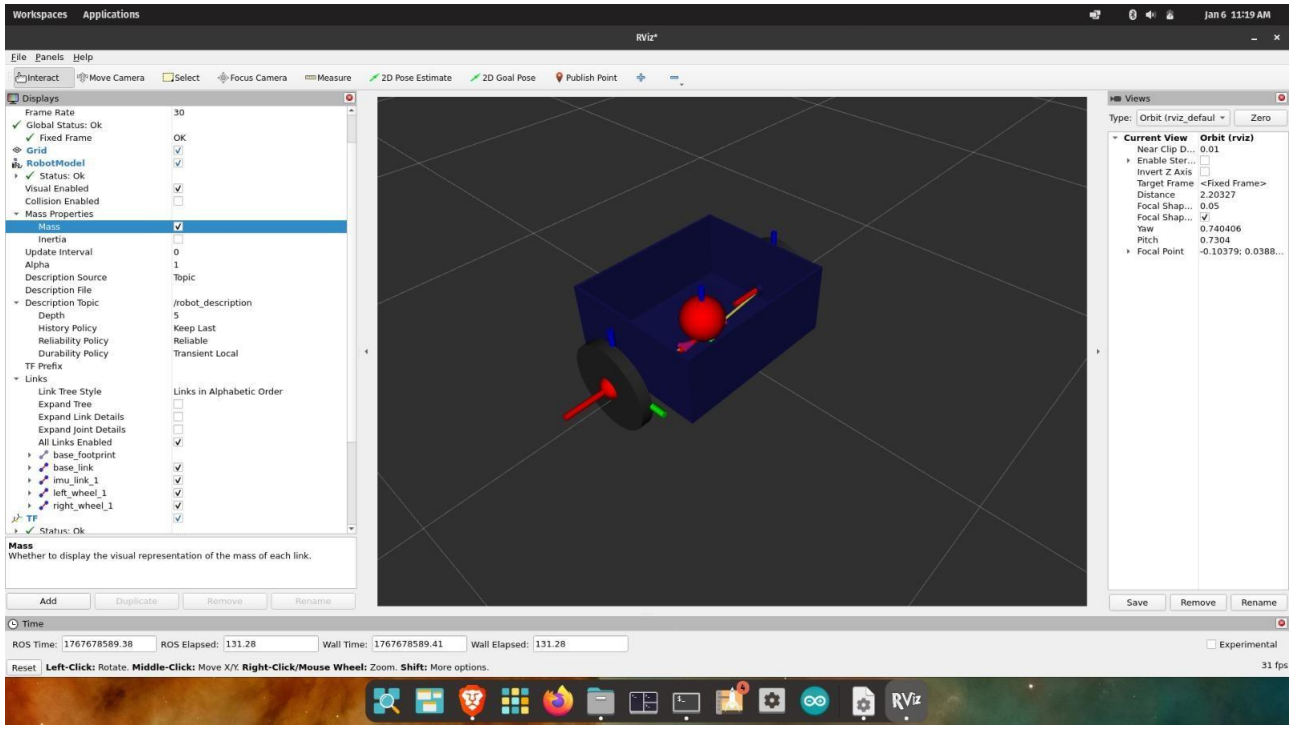


Figure 5: Visualizing CoM and Joints in rvs2

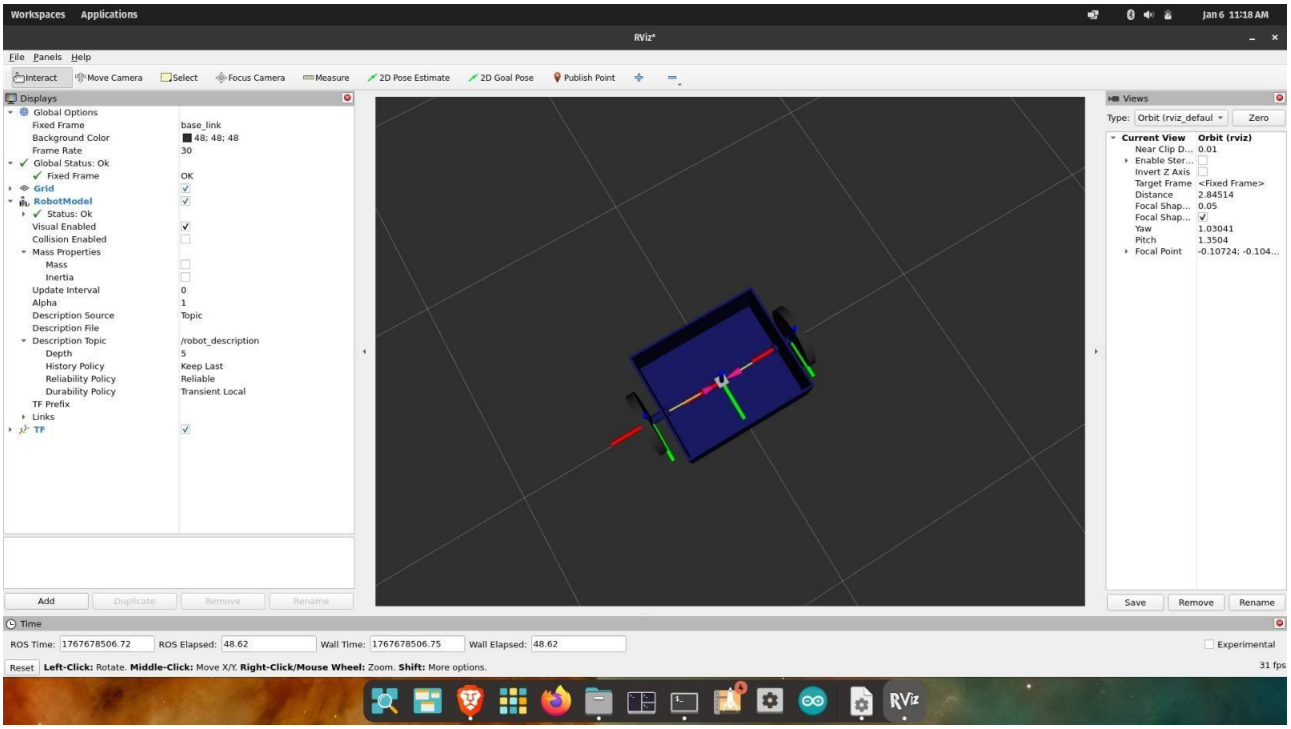


Figure 6: Visualizing CoM in rvs2

4.2.1 Launch File Architecture

A dedicated Python-based launch script was developed to orchestrate the initialization of the simulation environment. This launch file performs three critical functions:

Environment Configuration
The script dynamically sets the IGN_GAZEBO_RESOURCE_PATH environment variable to ensure that the Ignition Gazebo engine can locate the custom STL mesh files stored within the ROS 2 package share directory.

World Initialization
The custom simulation world file, ramps.sdf, is loaded during startup. This world contains a flat ground plane for controller tuning as well as inclined surfaces required to evaluate the robot's performance under varying gravitational and traction conditions.

Entity Spawning
The launch script utilizes the create node from the ros_ign_gazebo package to spawn the robot model into the simulation. The robot is initialized at a height of z = 0.2 m, allowing it to settle naturally onto the ground under gravitational forces, thereby avoiding unrealistic initial contact conditions.

4.3 Plugin Configuration

Standard URDF files describe only the geometric and kinematic structure of a robot. To enable sensing and actuation within the simulation, Ignition Gazebo model plugins were injected into the robot description using the self_balancing_bot_fusion.gazebo. gazebo file. These plugins interface directly with the DART physics engine to replicate real-world hardware behavior.

4.3.1 Ignition Gazebo IMU System

The primary feedback sensor for the self-balancing mechanism is the Inertial Measurement Unit (IMU). This sensor was simulated using the ignition::gazebo::systems:Imu plugin.

Configuration Details

Mounting
The IMU sensor was attached to the imu_link_1, which is rigidly fixed to the geometric center of the chassis. This placement ensures accurate measurement of body orientation and angular velocity.

Update Rate
A critical parameter for balance stability is the sensor update frequency. The plugin was configured to publish data at 100 Hz using the <update_rate>100</update_rate> tag. This provides orientation updates every 10 milliseconds, significantly reducing latency within the feedback control loop.

Data Topic
The plugin publishes raw orientation data in quaternion form along with angular velocity measurements to the internal Ignition topic /imu.

Visual Feedback
For debugging and validation purposes, the <visualize>true</visualize> tag was enabled, allowing the IMU coordinate frame to be rendered in the Ignition Gazebo graphical interface.

4.3.2 Differential Drive System

To actuate the robot, the Differential Drive plugin (ignition::gazebo::systems::DiffDrive) was implemented. This plugin functions as a virtual motor driver, converting high-level velocity commands into low-level joint torques applied within the physics engine.

Kinematic Parameters

Joint Mapping
The plugin was explicitly mapped to Revolute Joint 2 (Left Wheel) and Revolute Joint 1 (Right Wheel), ensuring correct torque application to the drive wheels.

Geometry
The kinematic constraints were configured to match the CAD model dimensions precisely:

- Wheel separation: 0.72 meters
- Wheel radius: 0.15 meters

Command Interface
The plugin subscribes to the /cmd_vel topic. At this integration stage, the robot listens for velocity commands but remains stationary, as no controller is yet publishing motion commands.

4.3.3 Physical Material Properties (Friction)

To prevent excessive slipping during balancing, realistic surface interaction properties were defined within the robot description.

Chassis
A low friction coefficient ($\mu = 0.2$) was assigned to the chassis surfaces. This allows smooth glancing collisions with the environment while avoiding unrealistic sticking behavior.

Wheels
A high friction coefficient ($\mu = 1.5$) was assigned to the wheel collision elements using the <mu1> and <mu2> parameters. This ensures effective ground traction, enabling the wheels to generate sufficient reaction torque to stabilize the inverted pendulum system.

4.4 ROS 2 Bridge Implementation

Ignition Gazebo and ROS 2 operate on different communication middleware systems (Ignition Transport and ROS DDS, respectively). To enable bidirectional data exchange, a ROS–Ignition Bridge (ros_gz_bridge) was implemented.

4.4.1 Establishing the Data Pipeline

The bridge was configured to translate selected topics, effectively linking the simulation environment with the ROS 2 computational graph.

Bridged Topics

Sensor Data (Gazebo to ROS 2)

- /Imu
  Transmits orientation and angular velocity data required for balance control.
- /Odom
  Transmits robot position and velocity data computed by the Differential Drive plugin.
- /Clock
  Synchronizes ROS simulation time with the Ignition Gazebo physics clock.

Control Signals (ROS 2 to Gazebo)

- /cmd_vel
  Receives velocity commands from the ROS 2 network and forwards them to the Differential Drive plugin to actuate the wheel joints.

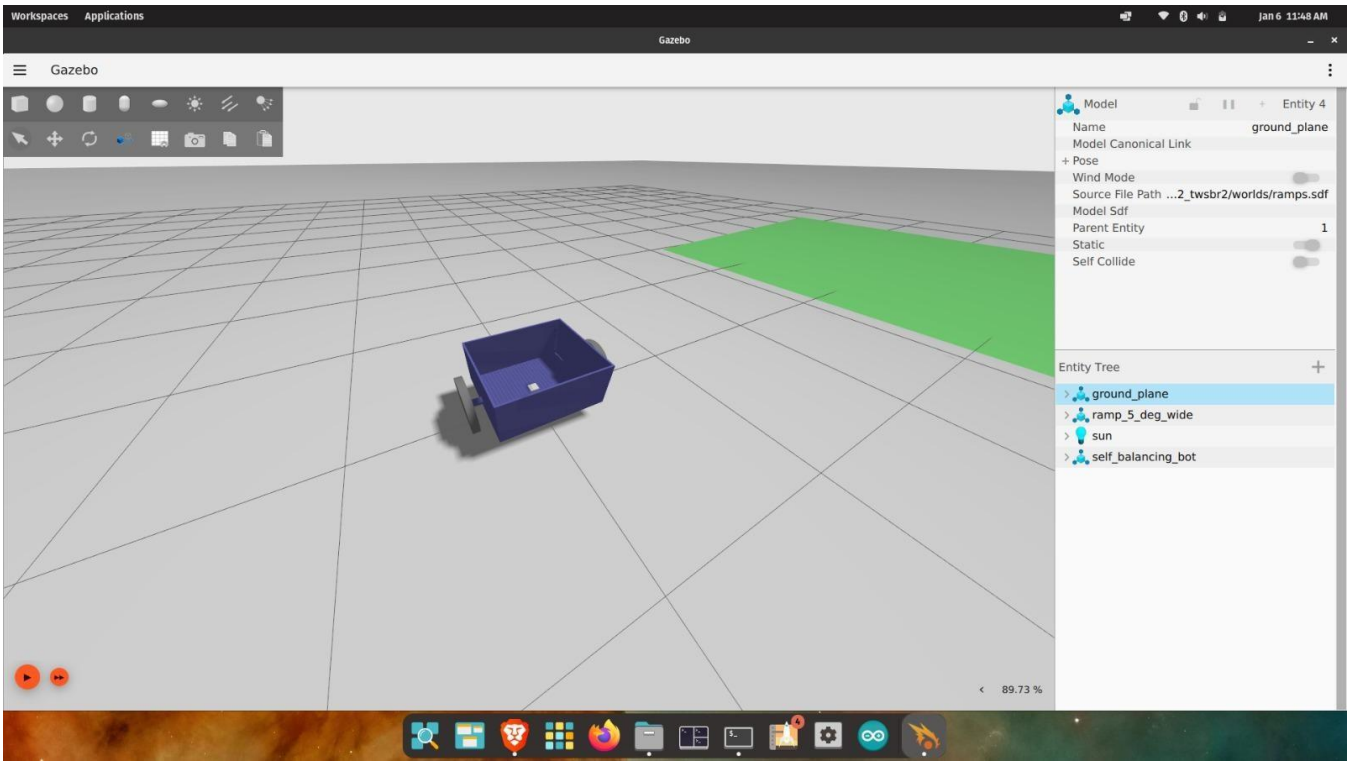


Figure 7: Spawned bot in Ignition Gazebo

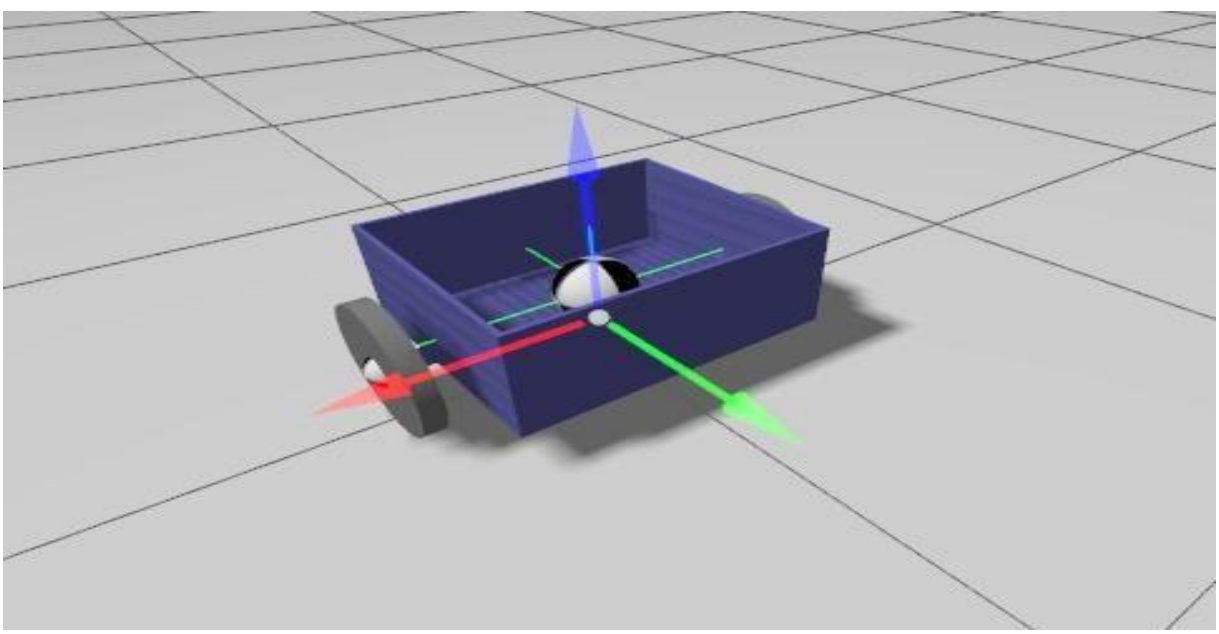


Figure 8: Visualizing CoM and joints and axis in Ignition Gazebo

4.5 ROS 2 Bridge Implementation

The final step in the simulation integration process involved establishing a robust communication pipeline between the Ignition Gazebo physics engine and the external control network. Ignition Gazebo and ROS 2 operate on different communication middleware systems: Ignition Gazebo uses Ignition Transport, while ROS 2 relies on DDS (Data Distribution Service). Due to this fundamental difference, the two systems cannot communicate directly.

To address this interoperability challenge, a ROS–Ignition Bridge was implemented. This bridge translates messages between Ignition Transport and ROS 2 DDS in real time, enabling seamless data exchange between the simulation environment and the external control architecture.

4.5.1 Establishing the MATLAB–ROS–Gazebo Control Loop

The ROS–Ignition Bridge was configured to create a bidirectional feedback loop connecting the physics simulation, the ROS 2 communication layer, and the MATLAB-based control system. Within this architecture, Ignition Gazebo functions as a "server" that publishes sensor data and listens for actuator commands, while MATLAB acts as the high-level decision-making unit.

This control loop is logically divided into two paths: the Sense path and the Act path.

1. The Sense Path (Gazebo → ROS 2 → MATLAB)

During simulation runtime, Ignition Gazebo continuously publishes the robot's physical state. The bridge forwards selected Ignition topics to ROS 2, which are then subscribed to by MATLAB.

IMU Feedback

Topic: /imu

Message Type: sensor_msgs/msg/Imu

This topic transmits orientation data in quaternion form along with angular velocity measurements generated by the simulated IMU sensor. This information constitutes the primary balance feedback required by the self-balancing control algorithm.

Odometry Feedback

Topic: /Odom

Message Type: nav_msgs/msg/Odometry

Published by the Differential Drive plugin, this topic provides the robot's global position (x, y) and linear velocity. Although not directly used for balancing, this data is essential for maneuvering logic, allowing the controller to determine forward motion and turning behavior.

2. The Act Path (MATLAB → ROS 2 → Gazebo)

The robot model in Ignition Gazebo is initialized in a passive state, with no actuation applied to the wheel joints. The Differential Drive plugin is configured to listen for control commands transmitted through the bridge.

Velocity Command Interface

Topic: /cmd_vel

Message Type: geometry msgs/msg/Twist

MATLAB publishes Twist messages containing Linear.x (forward velocity) and Angular.z

(yaw rate) commands. The ROS–Ignition Bridge listens for these messages, converts them into an Ignition-compatible format, and forwards them to the Differential Drive plugin.

Effect on the Simulation

Upon receiving a valid command, the plugin computes the required joint torques and applies them to the wheel joints in real time. If no command is received on the /cmd_vel topic, the wheels remain unactuated, and the robot behaves purely under the influence of gravity, friction, and collision forces, resulting in a natural fall of the inverted pendulum.

System Status at Integration Completion

At the conclusion of Chapter 4, the simulation environment is fully operational. The robot is successfully spawned in the ramps.sdf world with gravity, collisions, and friction enabled. Sensor data from the simulated IMU is actively published to the /imu topic on the ROS 2 network, and odometry information is available for navigation purposes.

The system is now fully prepared for closed-loop control. It awaits the introduction of the external "Brain," described in Chapter 5, which processes sensor feedback and generates motor commands to stabilize and maneuver the robot

CHAPTER 5
Software and Control System Design

5.1 Flowchart

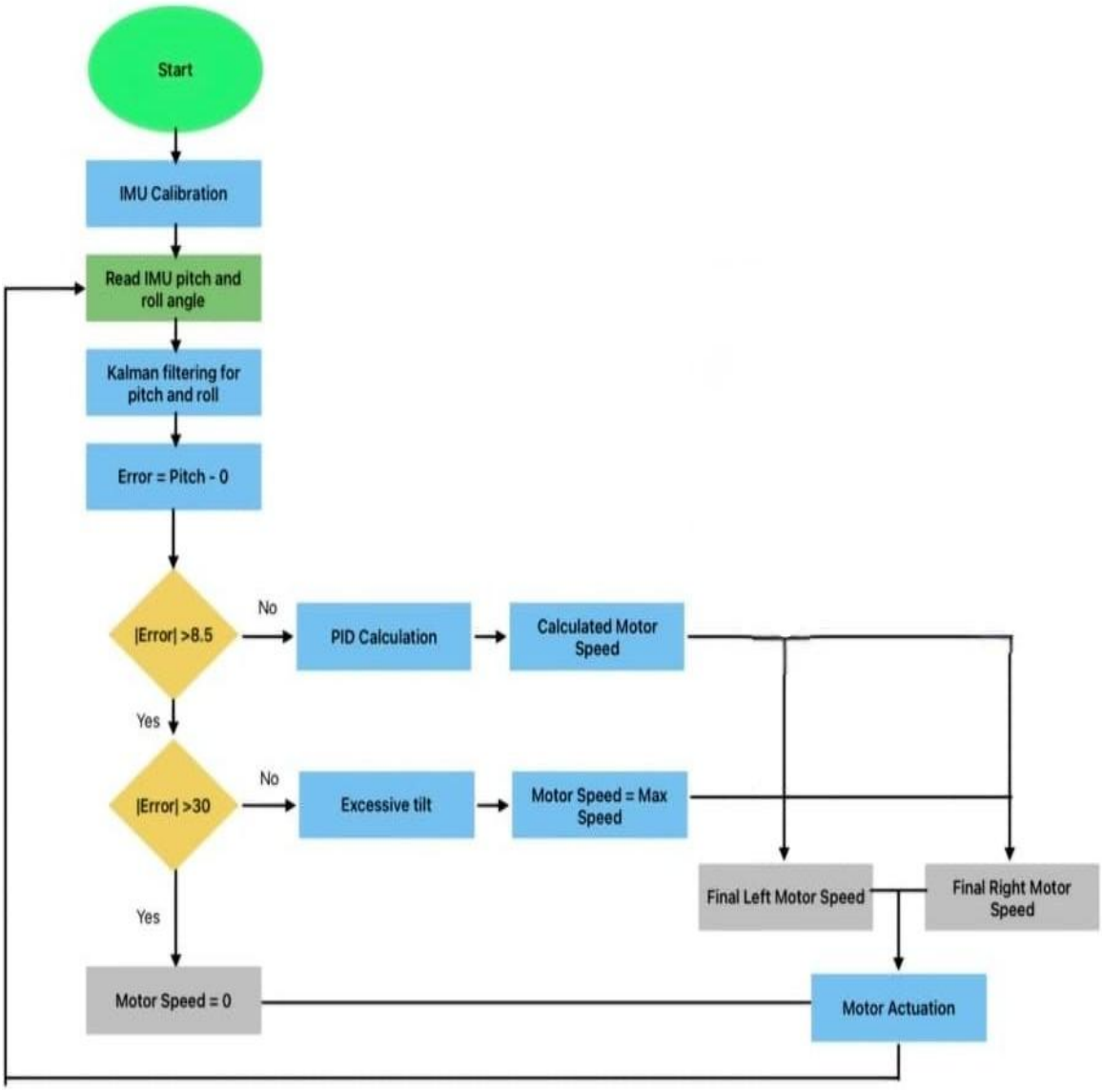
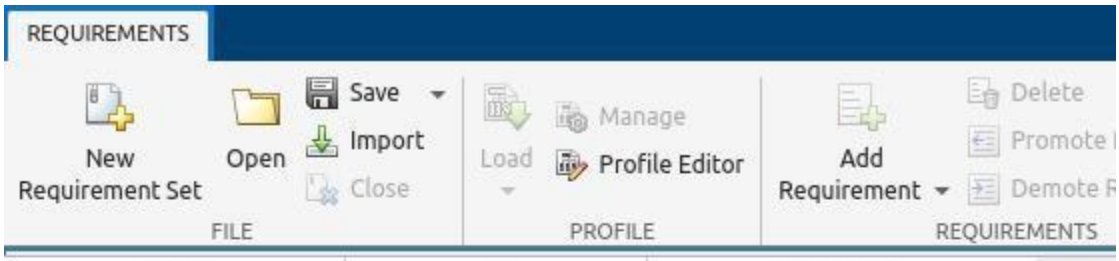


Figure 9: Flowchart

5.2: Requirements Collection in MATLAB



Index	ID	Summary
IMU		
1	Data Acquisition:	Must continuously read data...
2	Sensor Fusion:	Must implement a filtering al...
3	Axis Alignment:	ust be mounted and calibrat...
User		
1	Input Mechanism:	Must accept user inputs (via ...
2	Req_UI_01 (Maneuver...	* System shall accept and pr...
brain		
1	PID Algorithm:	Must implement a Proportion...
2	Input Processing:	Must calculate the error: Erro...
3	Safety Cutoff:	Must detect if the robot has f...
motor driver		
1	ignal Amplification:	Must accept low-current logi...
2	Direction Control:	Must utilize an H-Bridge topo...
motor		
1	Traction:	Wheels/Tires attached to the...
2	Symmetry:	Both motors must have very...

Table 1: Requirements Collection in MATLAB

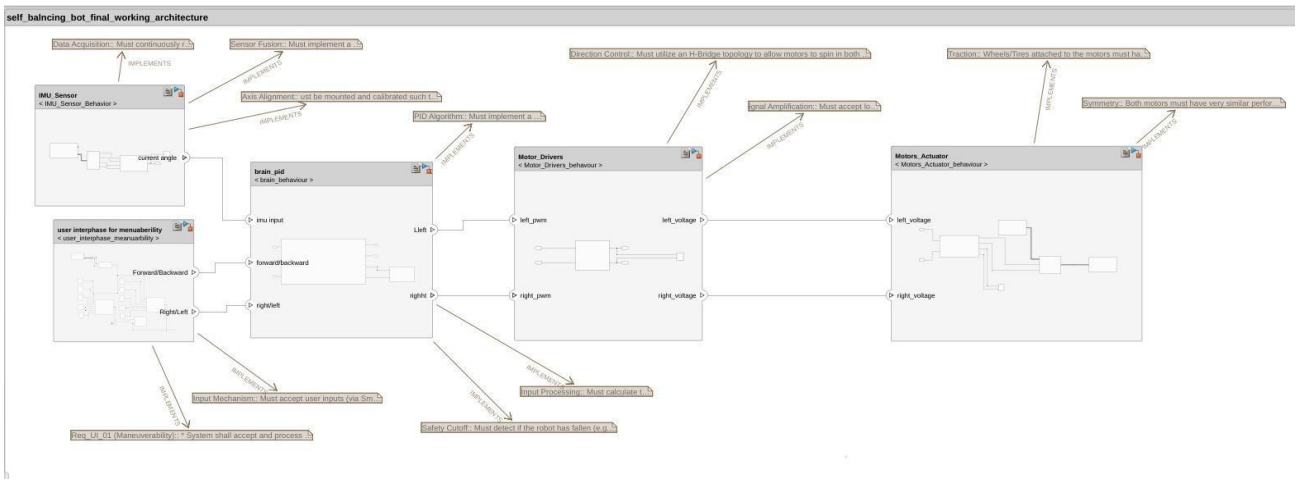


Figure 10: System Design Linked to Requirements In MATLAB

5.1 Control System Architecture

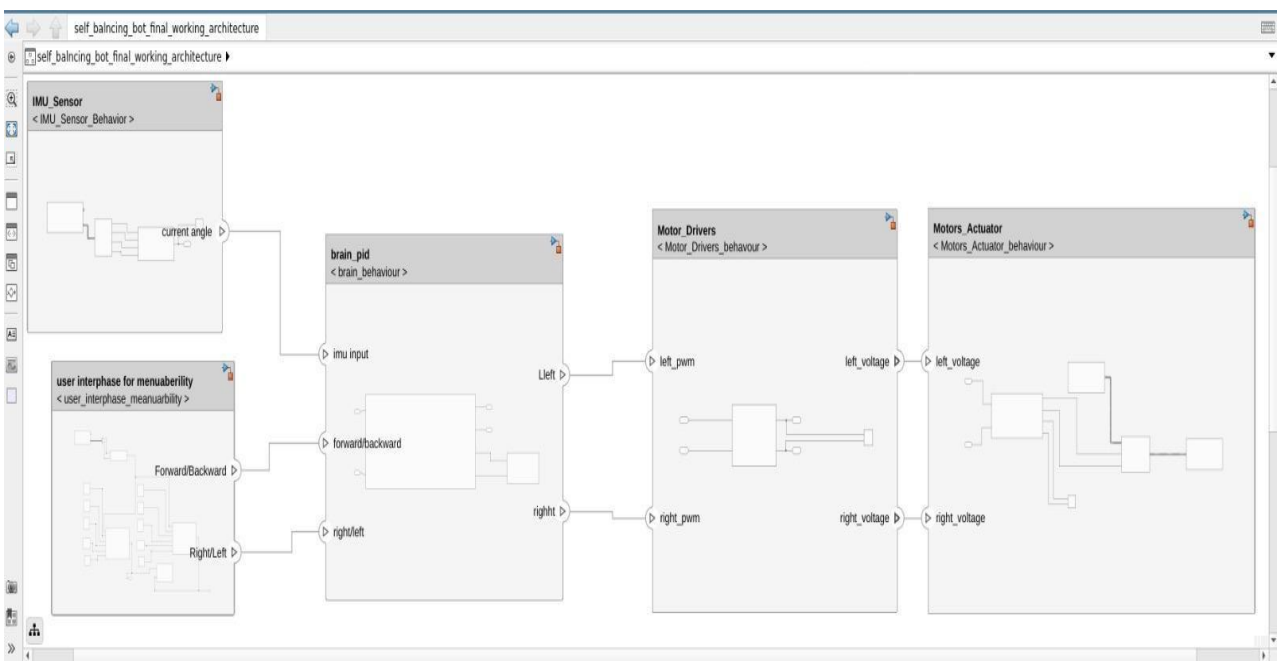


Figure 11: System architecture

5.2 MATLAB/Simulink

The control logic of the self-balancing robot is centralized within the MATLAB Simulink environment, which functions as the “Brain” of the system. This high-level architecture was

intentionally designed to be modular, enabling clear separation between sensor processing, decision-making algorithms, and actuator command generation. Such modularity simplifies debugging, tuning, and future expansion of the control strategy.

Architecture Overview

The primary Simulink model, named self_balancing_bot_final_working_architecture,
is composed of four major subsystems connected in series:

IMU_Sensor
Responsible for subscribing to sensor data from the ROS 2 network and processing it into a usable physical state representation.

User_Interface (Maneuverability)
Handles human-in-the-loop control logic, including keyboard-based WASD inputs for forward, backward, and turning commands.

Brain_PID
Implements the core stabilization algorithm responsible for maintaining the robot in its upright position.

Motor_Drivers
Converts the controller output into actuator-ready commands, such as PWM duty cycles or equivalent voltage signals for wheel motors.

This layered design ensures that changes to one subsystem (for example, switching from PID to LQR control) do not require modifications to the entire model.

5.3 State Estimation (IMU Data Processing Subsystem)

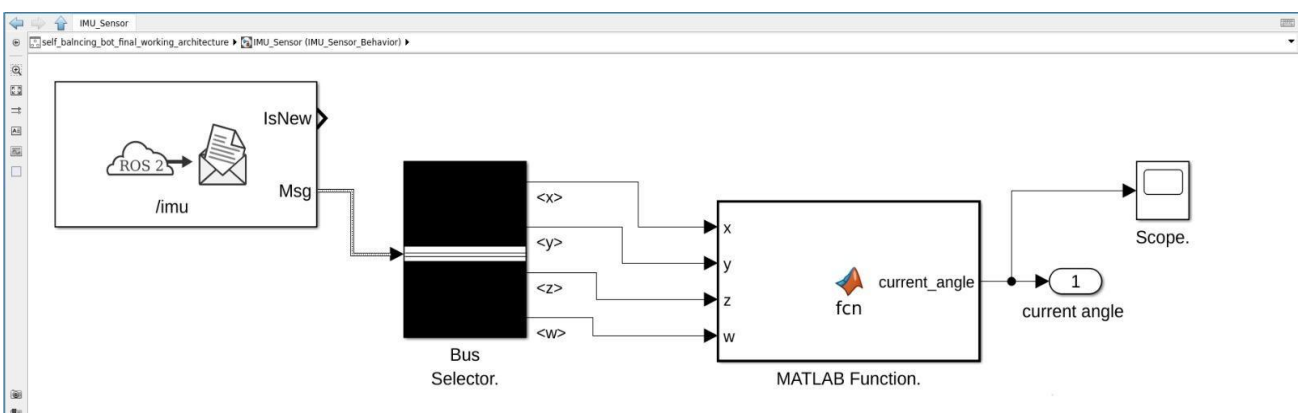


Figure 12: IMU Subsystem

The first and most critical stage of the closed-loop control system is the IMU_Sensor subsystem. This block is responsible for acquiring raw sensor data from the ROS 2 network and converting it into a meaningful state variable: the robot's tilt angle, denoted by θ. Accurate estimation of this angle is essential for stable balancing control.

5.3.1 Data Acquisition (ROS Subscribe)

The entry point of the IMU_Sensor subsystem is a ROS 2 Subscribe block configured as follows:

Topic: /imu
Message Type: sensor_msgs/Imu

Function
This block listens to the Ignition Gazebo simulation in real time. At every simulation update (100 Hz), it captures the latest IMU message containing the robot's orientation and angular velocity. This ensures that the control loop operates on up-to-date sensor information with minimal latency.

5.3.2 Signal Decomposition (Bus Selector)

The ROS IMU message is a structured data object (Bus) containing multiple fields. To extract only the required information, a Bus Selector block is used.

Input
The raw /imu ROS message.

Selected Signals
The Orientation field is isolated, which contains the quaternion components representing the robot's orientation in three-dimensional space:

- x, y, z (vector components)
- w (scalar component)

These four values form the quaternion representation of orientation.

5.3.3 Quaternion to Euler Angle Decoding (Custom MATLAB Function)

The orientation data published by Ignition Gazebo is expressed in quaternion form $q = w + xi + yj + zk$.

Quaternions are used to avoid singularities such as gimbal lock, but they are not directly usable by classical controllers such as PID, which require a single angular measure.

For a self-balancing robot, the required variable is the pitch angle (rotation about the Y-axis), expressed in degrees. Standard conversion blocks often output angles in radians or exhibit discontinuities near ±90°, which can destabilize the controller. To address this, a Custom MATLAB Function block was implemented.

Mathematical Logic

The pitch angle θ is computed from the quaternion components using the standard conversion formula:

$$\theta = \arcsin(2(wy - zx))$$

The result is then converted from radians to degrees.

CODE

```
function current_angle = fcn(x, y, z, w)

    % Convert Quaternion to Pitch (Rotation around Y-axis)

    % Result is in Degrees

    sinp = 2 * (w * y - z * x);

    current_angle = asin(sinp) * (180/pi);

end
```

Output

The output of this subsystem is a single scalar signal named current angle. This value represents the robot's instantaneous tilt angle in degrees, where 0° corresponds to a perfectly upright position. This clean, noise-processed signal is then fed directly into the Brain_PID subsystem for stabilization control.

5.4 Subsystem 1: IMU

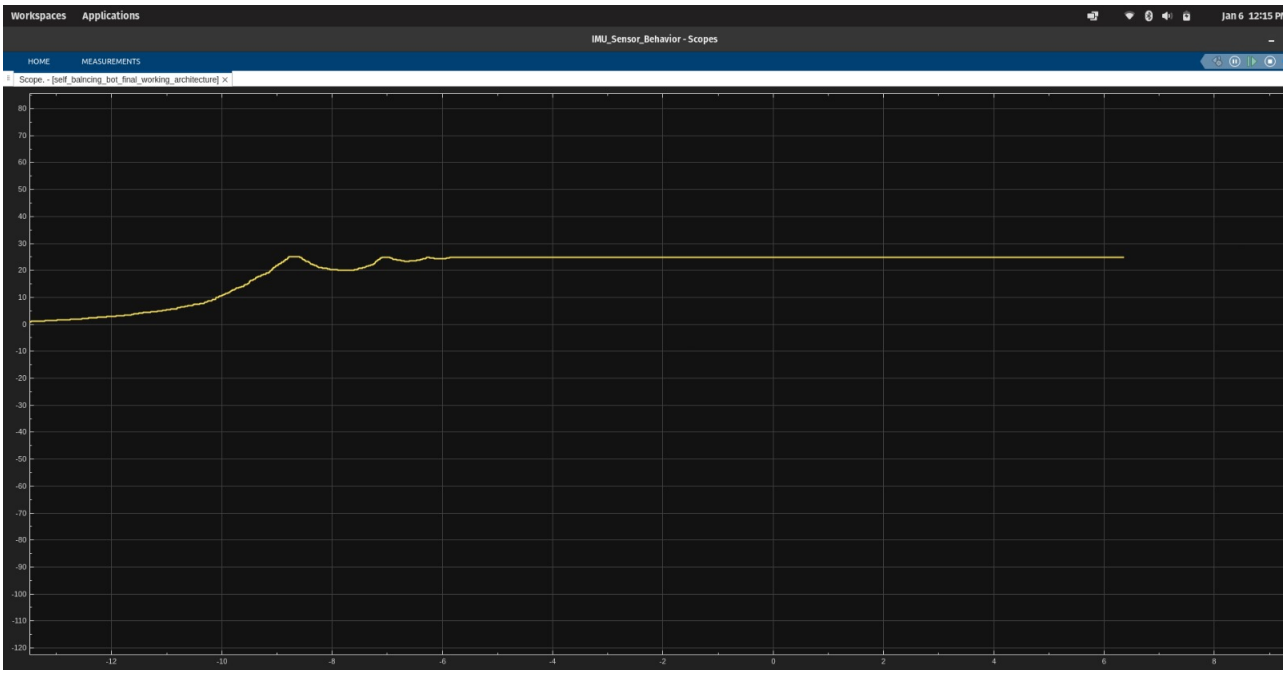


Figure 13: IMU graph Data visualization tilt

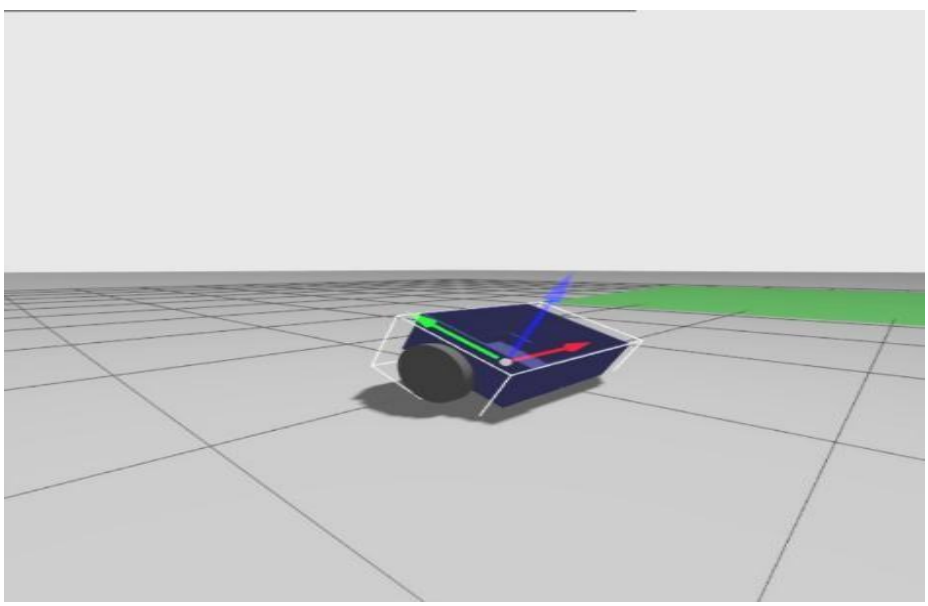


Figure 14: Bot leaning over due to gravity when PID is OFF

5.4.1 Unit Testing: IMU Data Verification

Before attempting active balancing, it was essential to validate the integrity of the sensor

feedback loop. If the *"Brain"* of the system receives incorrect or delayed angle data, even a perfectly tuned PID controller will fail. Therefore, an open-loop free-fall test was conducted.

Objective

To verify that the Gazebo → ROS 2 → MATLAB data pipeline was functioning correctly and that the Quaternion-to-Euler angle conversion logic was accurate.

Test Setup

- Controller State:
  The PID controller gains were set to zero

$K_p = 0, K_i = 0, K_d = 0$

effectively disabling the actuation subsystem.

- Initial Condition:
  The robot was spawned in Gazebo in a nearly upright position with an initial height of approximately

$z \approx 0.2\,\mathrm{m}$

Procedure

The simulation was started, allowing gravity to act on the robot freely without any motor resistance or corrective control action.

Results

Angle vs Time Graph (PID OFF / Free Fall)]

Observation and Analysis

Figure 6.1 illustrates the robot's tilt angle versus time during the free-fall test. The response can be divided into three distinct regions:

- Region A – Initial Upright State:
  The tilt angle begins near 0°, indicating a stable upright posture.
- Region B – Free Fall:
  The angle rapidly diverges from zero as the robot topples. The slope of the curve represents the angular velocity during the fall.
- Region C – Ground Impact:
  The angle saturates near $\pm 90^\circ$(depending on the direction of fall) once the chassis contacts the ground.

Conclusion

The smooth and continuous nature of the angle response confirms that the IMU subsystem is functioning correctly. The sensor accurately detected real-time orientation changes, and the MATLAB decoding block successfully converted quaternion data into a meaningful pitch angle.

This successful free-fall validation test confirms the correctness of the input stage of the control loop, thereby clearing the way for closed-loop PID tuning and active balancing experiments.

5.5 Subsystem 2: User interface

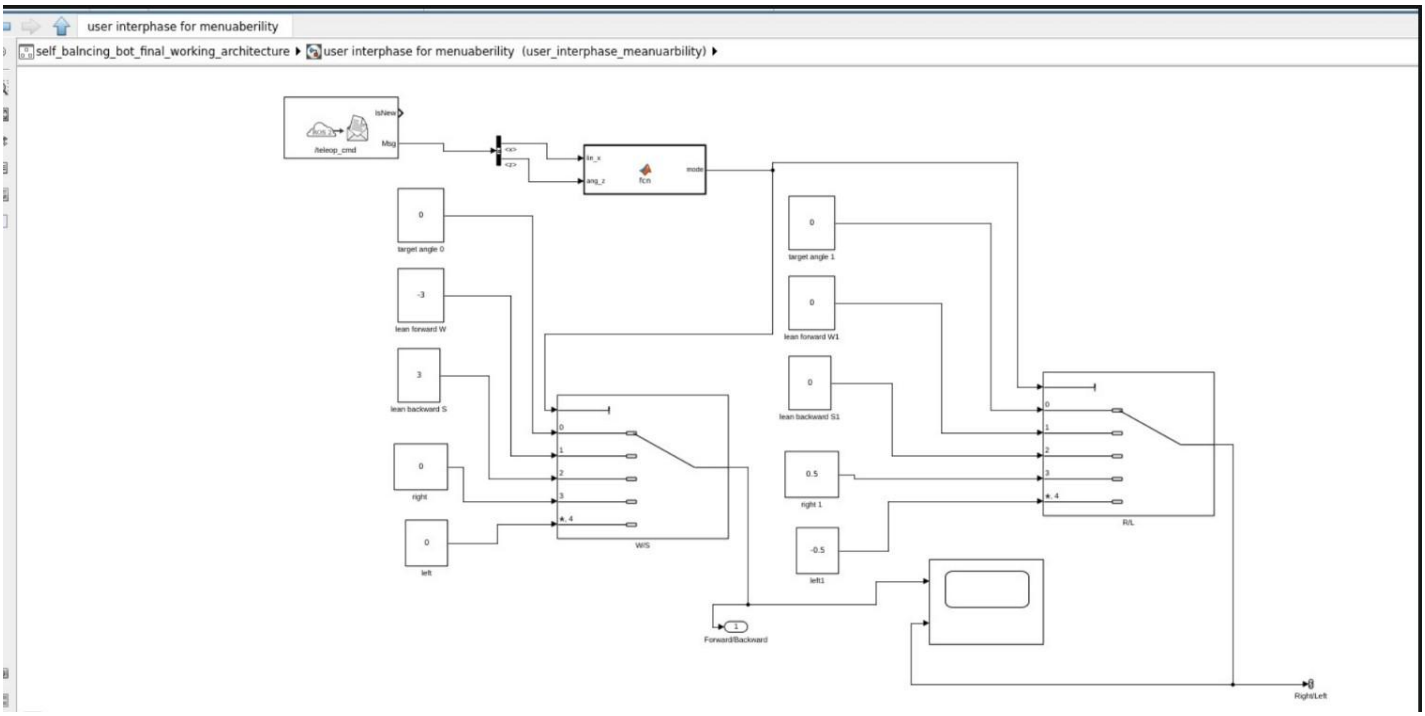


Figure 15: User Interface

The maneuverability Subsystem acts as the interface between the human operator and the robot's self-balancing control algorithm. Unlike a conventional wheeled vehicle—where forward motion is achieved by directly commanding wheel velocities—a self-balancing robot must deliberately shift its equilibrium point to initiate motion.

This subsystem intercepts user commands and modifies the balance setpoint, effectively causing the robot to lean in a controlled manner. The balancing controller then reacts to this intentional imbalance, resulting in smooth and stable motion

5.5.1 Input Interface (Teleoperation)

The control chain begins with a ROS 2 Subscribe block, which receives user commands from a teleoperation node.

- Topic: /teleop_cmd
- Source: A Python-based terminal script running on the host machine, capturing keyboard inputs (W, A, S, D).
- Signal Extraction:
  The incoming message is routed through a Bus Selector, which extracts:
  - Linear.x – Forward/Backward command

- Angular.z – Left/Right turning command

These two signals form the raw user intent inputs to the maneuverability logic.

5.5.2 Logic Parsing (Custom MATLAB Function Block)

The extracted velocity commands are passed to a MATLAB Function Block (fcn), which functions as a state classifier. Rather than forwarding raw velocity values to the controller, this block converts user intent into a discrete Mode ID.

The output Mode is defined as:

- Mode 0: Idle / Stop
- Mode 1: Forward (W)
- Mode 2: Backward (S)
- Mode 3: Turn Right (D)
- Mode 4: Turn Left (A)

This discrete representation simplifies downstream logic and enables deterministic switching behavior.

5.5.3 Target Angle Manipulation (Forward/Backward Logic)

Linear motion is implemented using a Multiport Switch (labelled *W/S* in the Simulink diagram). This switch selects the target pitch angle for the balancing PID controller based on the decoded Mode ID.

- Stationary (Mode 0):
  Target angle = 0°
  The robot attempts to maintain a perfectly upright posture.
- Forward Command (Mode 1 – 'W'):
  Target angle = −3°

  Physical Interpretation:
  By setting the reference angle to −3°, the controller perceives the robot as leaning too far backward relative to its desired orientation. To correct this perceived error, the PID controller commands the wheels to roll forward, causing continuous forward motion while maintaining balance.

- Backward Command (Mode 2 – 'S'):
  Target angle = +3°

Physical Interpretation: The robot is commanded to lean forward. The controller responds by driving the wheels backward to reposition the base under the centre of mass.

This mechanism allows translational motion to be achieved purely through equilibrium manipulation.

5.5.4 Differential Drive Mixing (Turning Logic)

Rotational motion is handled independently using a second Multiport Switch (labelled *R/L*). Unlike the pitch logic, this block does not affect balance; instead, it produces a velocity bias for differential steering.

- Turn Right (Mode 3 – 'D'): Output bias = +0.5
- Turn Left (Mode 4 – 'A'): Output bias = −0.5
- No Turn (Mode 0, 1, 2): Output bias = 0

This bias is added to one motor command and subtracted from the other, producing yaw rotation while preserving pitch stability.

Figure 16: User interface in terminal

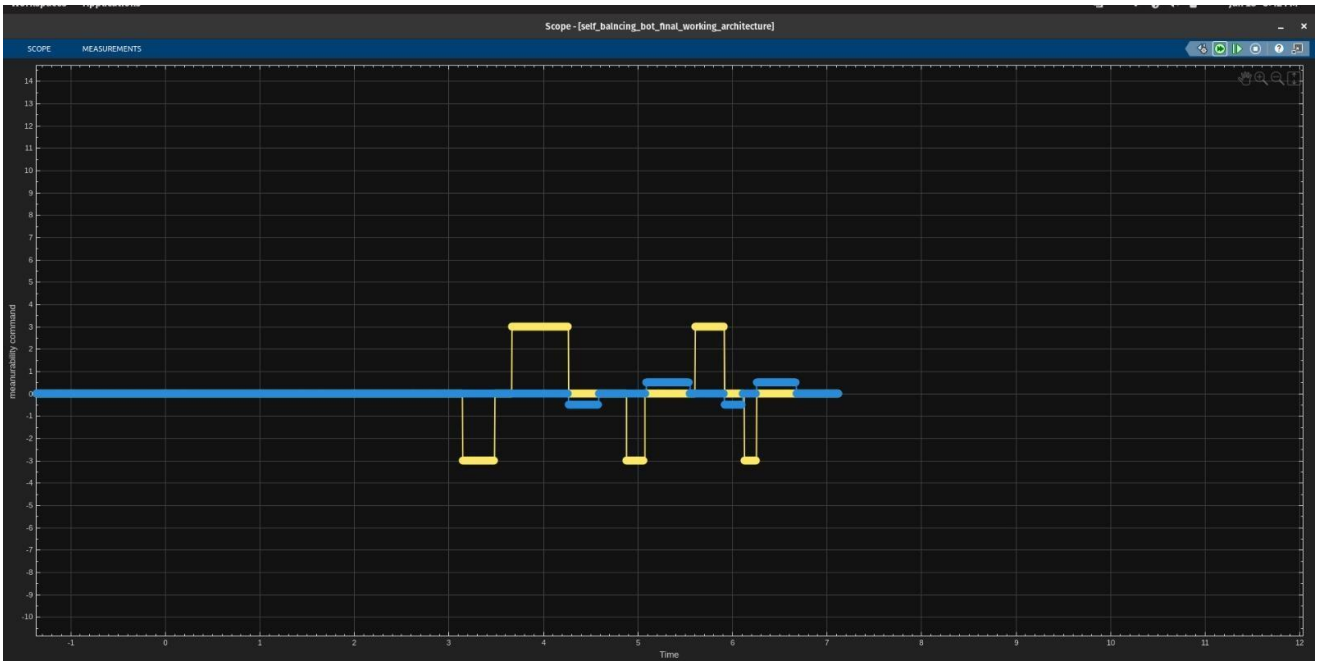


Figure 17: User interface graph

5.6 Subsystem 3: PID

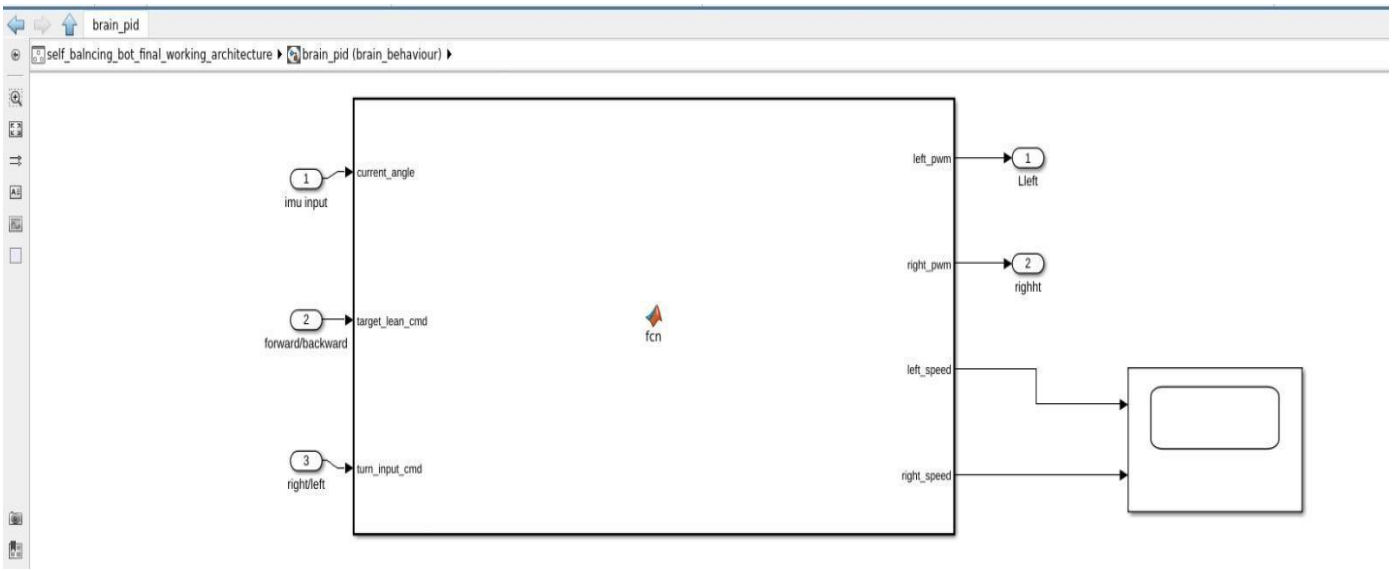


Figure 18: PID subsystem

5.6.1 PID Controller Implementation (The "Brain" Subsystem)

The core of the robot's stability and motion control is implemented within the Brain_PID subsystem. This block functions as the robot's central processing unit, combining real-time sensor feedback and user intent to compute the final motor drive commands.

5.6.2 Input–Output Architecture

As illustrated in Figure 5.4, the Brain_PID subsystem is implemented as a central MATLAB Function block with a carefully designed input–output interface that simultaneously supports balance control and maneuvering.

Inputs

- current angle
  Real-time pitch angle θ obtained from the IMU subsystem.
- target_lean_cmd
  Desired equilibrium angle generated by the maneuverability logic (e.g., -3° for forward motion).
- turn_input_cmd
  Rotational bias command used for steering (e.g., 0.5).

Outputs

- left_pwm, right_pwm
  Final motor control signals expressed as PWM values, ranging from

$$-250 \leq \text{PWM} \leq +250$$

These signals are sent to the Motor Driver subsystem to regulate the effective voltage applied to the motors.

5.6.3 Control Algorithm (Internal Logic)

Rather than a conventional linear PID controller, the Brain_PID subsystem implements a Switching PD Controller with a multi-stage processing pipeline. This architecture ensures that the physical requirement of balancing speed is accurately translated into safe and effective electrical control signals.

Stage 1: Signal Smoothing (Slew Rate Limiter)

Directly applying a step change in target angle (e.g., from 0° to -3°) can induce jerks and destabilize the system. To mitigate this, a slew rate limiter is applied to the target angle.

Logic:

The controller computes the difference between the current internal target and the requested target. If the difference exceeds a defined threshold, the target is incremented gradually.

Constraint:

$\Delta\theta < 0.05^\circ$ per control cycle

Result:

When the user presses 'W', the robot leans forward smoothly over several cycles, producing natural and mechanically safe motion.

Stage 2: Error Calculation (Balancing Physics)

The fundamental balancing action is governed by the pitch error term:
$e(t) = \theta_{target} - \theta_{current}$

Stationary Balancing
If $\theta_{target} = 0^\circ$ and the robot tilts forward to $+5^\circ$:
$e(t) = -5^\circ$

The controller responds by driving the wheels forward to counteract the fall.
Forward Motion
To move forward, the target angle is set to:
$\theta_{target} = -3^\circ$

Even when the robot is physically upright ($\theta_{current} = 0^\circ$), the error becomes:
$e(t) = -3^\circ$

The controller interprets this as a falling condition and accelerates the wheels forward, producing controlled motion.

Stage 3: Velocity Calculation (PD Control Law)
At this stage, the controller calculates the required wheel velocity needed to correct the error. The corrective control effort represents the physical speed the robot must achieve to regain balance.

$$u_{\text{speed}}(t) = K_p \cdot e(t) + K_d \cdot \frac{de(t)}{dt}$$

Controller Gains:

- Proportional Gain:

$K_p = 18.0$

Provides strong corrective velocity proportional to the tilt angle.

- Derivative Gain:

$K_d = 2.5$

Acts as a virtual damper by reacting to the rate of change of error, reducing overshoot.

Stage 4: Speed-to-PWM Conversion

The calculated u_{speed} is a theoretical velocity value, whereas the motor drivers require a digital PWM signal. Therefore, the system maps velocity to a PWM duty cycle.
Mapping Logic:
$\text{PWM}_{output} = \text{map}(u_{\text{speed}}, \text{min_speed}, \text{max_speed}, -250, 250)$

Saturation:
The PWM signal is strictly clamped within the range $[-250, +250]$, ensuring that motor commands never exceed driver limits and protecting the H-bridge from over-current conditions.

5.6.4 Maneuverability and Turning Logic

Turning in a self-balancing robot introduces additional instability due to centrifugal forces. To address this, a Stability Manager is embedded within the control logic to ensure that balance is never sacrificed for maneuverability.

5.6.5. Stability Manager (Rescue Mode vs. Maneuver Mode)

The controller continuously monitors the robot's deviation from the target angle:

$$\text{Deviation} = | \theta_{current} - \theta_{target} |$$

Rescue Mode (Unstable State)

- Condition:

$$\text{Deviation} > 3.0°$$

- Action:
  - 100% of PWM authority is dedicated to balancing
  - Turning command is forcibly set to zero

Safety Rationale:

This prevents the user from commanding turns while the robot is recovering from a large disturbance, which could otherwise cause a fall.

Maneuver Mode (Stable State)

- Condition:

$$\text{Deviation} \leq 3.0°$$

- Action:
  - Balance output is attenuated:

$$\text{PWM}_{final} = \text{PWM}_{balance} \times 0.6$$

  - Remaining motor authority is reserved for turning.

5.6.6 Differential Drive Mixing (Left / Right Logic)

When operating in Maneuver Mode, steering is achieved using differential drive mixing:

$$\text{PWM}_{Left} = \text{PWM}_{Balance} + \text{PWM}_{Turn}$$

$$\text{PWM}_{Right} = \text{PWM}_{Balance} - \text{PWM}_{Turn}$$

Right Turn Example:

If $\text{PWM}_{Balance} = 100$ and $\text{PWM}_{Turn} = 50$:

- Left Motor = 150
- Right Motor = 50

The robot arcs smoothly to the right while maintaining balance.

Zero-Radius Turn:

If $\text{PWM}_{Balance} \approx 0$, the wheels spin in opposite directions ($+50/-50$), allowing the robot to rotate in place.

5.6.7 Closed-Loop Stability Test (PID Active)

Once the PD controller was activated using the tuned parameters
$K_p = 18.0, K_d = 2.5$

the self-balancing system successfully achieved stable operation.

Analysis
In contrast to the open-loop free-fall experiment, the closed-loop response in Figure 6.2 demonstrates a well-regulated equilibrium. The robot's tilt angle remains cantered near 0°, with small oscillations confined within approximately $\pm 1^\circ$.
These oscillations represent the continuous corrective action of the PD controller as it compensates for gravity, sensor noise, and dynamic coupling effects. The absence of large overshoot or divergence confirms that the proportional gain provides sufficient corrective authority, while the derivative term effectively damps oscillations.
Conclusion
This experiment validates the effectiveness of the tuned PD controller in maintaining upright balance. The system exhibits stable closed-loop behaviour with minimal steady-state error, confirming correct implementation of the Brain_PID subsystem.

5.6.8 Control Effort Analysis (PWM Response)
The PWM response graph illustrates the control effort generated by the Brain_PID subsystem during active balancing.

Analysis of Speed-to-PWM Translation
The graph visualizes the final output of the Stage 4 speed-to-PWM conversion process.

- High-Frequency Switching:
  Rapid oscillations between positive and negative PWM values indicate continuous micro-corrections required to stabilize the inverted pendulum system.
- Velocity Tracking:
  Each fluctuation represents the PD controller calculating a required corrective velocity and immediately converting it into a high-current PWM burst.
- Aggressive Yet Bounded Response:
  Large PWM spikes (e.g., from 0 to +200) occur when the robot begins to drift, demonstrating fast corrective action. The bounded nature of the signal confirms that the velocity-to-PWM mapping is correctly calibrated for the motor's power curve.

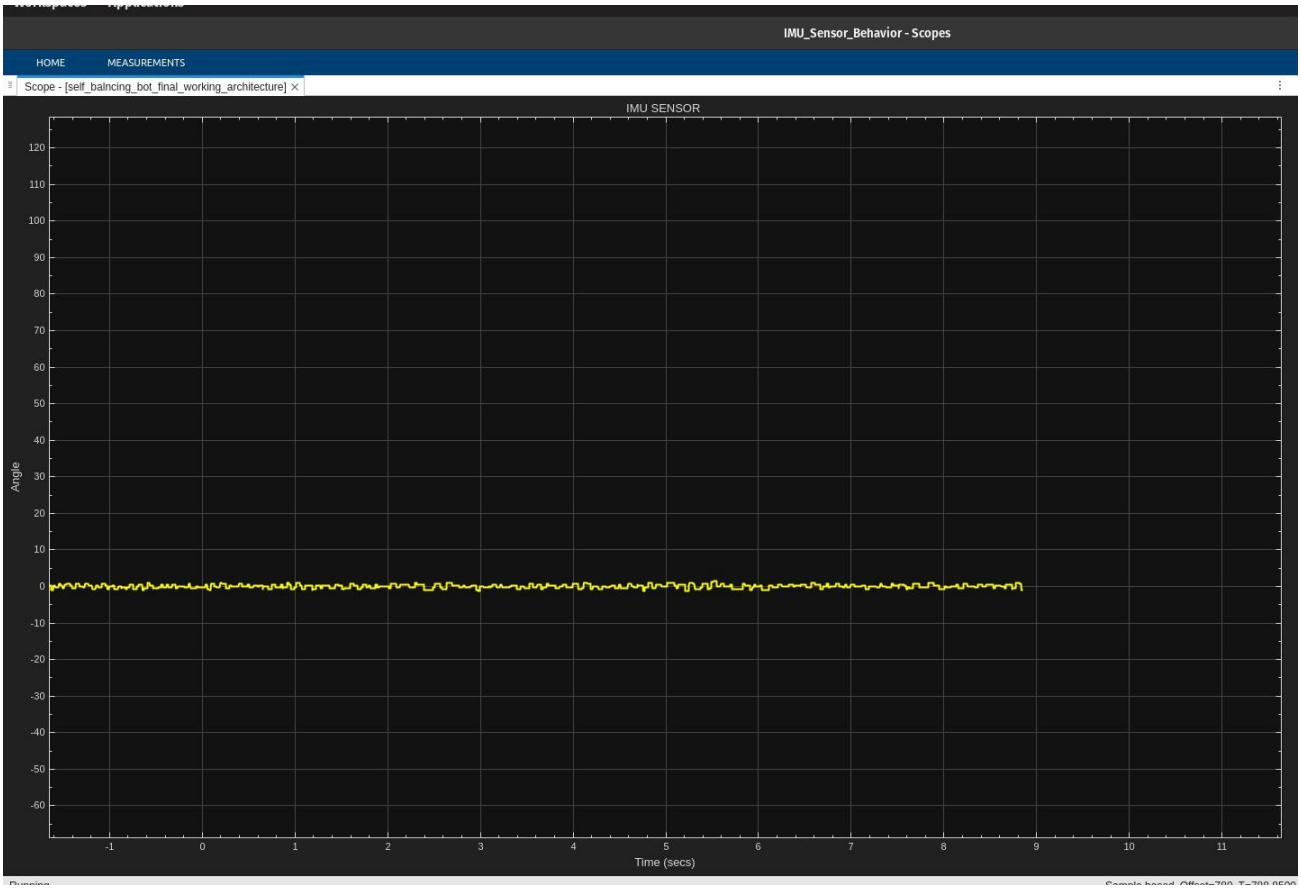


Figure 19: Imu Data after tunning PID

Control Effort Analysis (PWM Response)

The corresponding PWM response illustrates the control effort generated by the Brain_PID subsystem to maintain upright stability. The rapid oscillations in the motor command signals—frequently switching between positive and negative values—indicate continuous, high-frequency torque corrections applied by the motors.

This behavior is characteristic of a well-tuned balancing controller. Each fluctuation represents an instantaneous corrective action in response to small deviations in tilt angle. When the robot begins to drift away from equilibrium, the controller immediately applies counter-torque to arrest the motion before it can amplify.

The aggressive yet bounded nature of the PWM output confirms that the controller reacts quickly without saturating the actuators. As a result, minor angular disturbances are effectively suppressed, preventing them from growing into large deviations or a complete loss of balance.

Figure 20: PWM Data

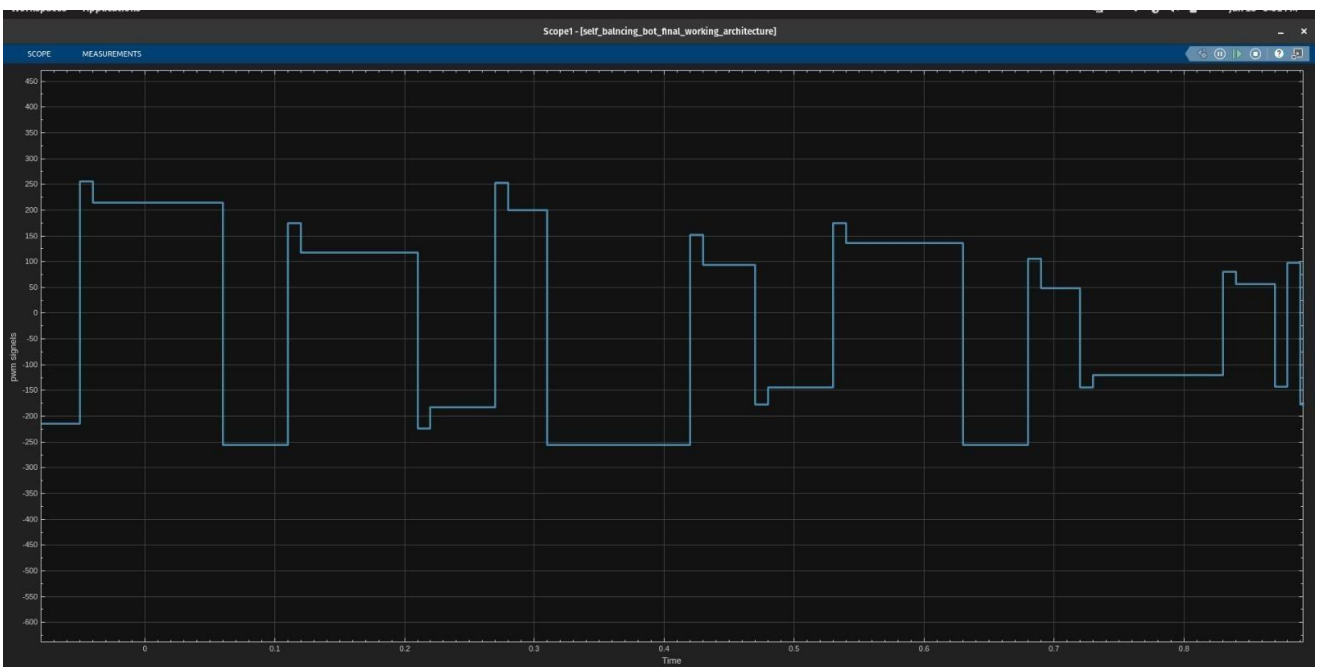


Figure 21: PWM signal

5.7 Subsystem 4: Motor Driver

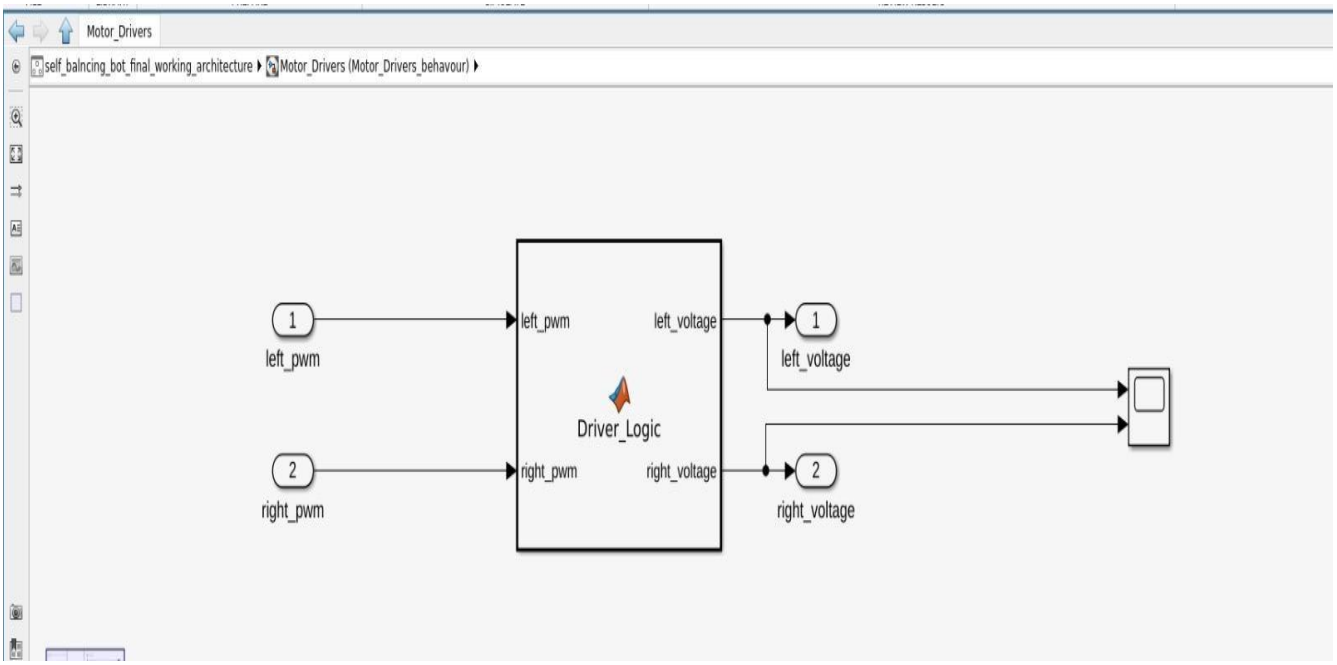


Figure 22: Motor Driver Subsystem

5.7.1 Input Signals (Control Effort)

The Motor Drivers subsystem receives two control effort signals from the upstream Brain_PID block:

- left_pwm – Control effort for the left motor
- right_pwm – Control effort for the right motor

These signals are dimensionless effort values produced by the PD controller. In theory, PID outputs can range from $-\infty$ to $+\infty$, where:

- Sign (+ / −):
  Indicates the desired direction of rotation (Positive = Forward, Negative = Reverse)
- Magnitude:
  Represents the intensity of the corrective torque required to maintain balance.

Because real hardware cannot deliver infinite voltage or current, these signals must be conditioned before being applied to the motors.

5.7.2 Driver Logic (Virtual H-Bridge Model)

The core processing occurs inside the Driver_Logic MATLAB Function block, which models the behaviour of common motor drivers such as the L298N or BTS7960.

Mathematical Implementation

A saturation function is applied to clamp the raw PID output to realistic voltage limits defined by the simulated power supply:

$$V_{out} = \begin{cases} V_{\max}, & u(t) > V_{\max} \\ u(t), & -V_{\max} \le u(t) \le V_{\max} \\ -V_{\max}, & u(t) < -V_{\max} \end{cases}$$

where:

- $u(t)$is the raw PID control effort
- $V_{\max}$is the maximum allowable motor voltage (e.g., 5.0)

Operational Logic

- If the PID controller requests an aggressive correction (e.g., $u(t) = 8.5$) during a near-fall condition, the driver logic clamps the output to $V_{\max}$.
- This ensures the commanded voltage remains within physically realizable limits.

Purpose

- Prevents numerical instability or unrealistic force application in simulation
- Accurately models the voltage ceiling imposed by real battery-powered motor drivers
- Protects downstream actuator models from saturation-induced instability

5.7.3 Output Signals (Actuator Commands)

The Motor Drivers subsystem produces two final, conditioned outputs:

- left voltage
- right voltage

These signals are clean, bounded, and safe representations of motor voltage commands. They are passed to the Motors Actuator block, which formats them into the appropriate ROS 2 message structure (geometry_msgs/Twist) required by the Gazebo Differential Drive Plugin. This final step converts the control voltages into physical wheel torques, completing the control loop from sensor perception to mechanical actuation.

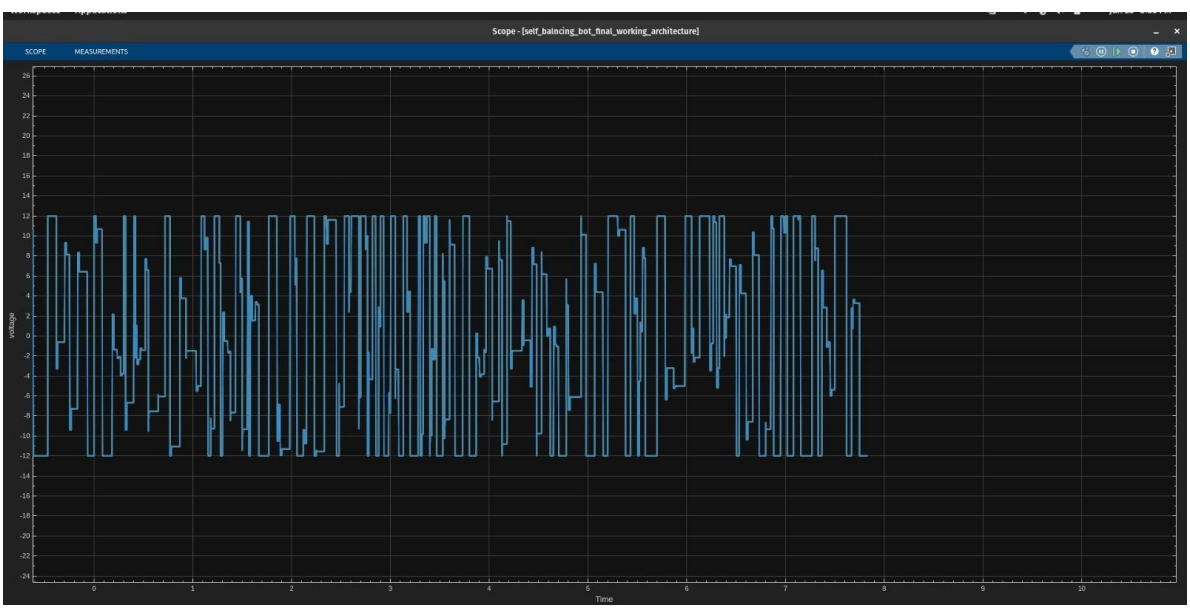


Figure 23: Motor driver converting PWM signal into 12 v

5.8 Subsystem Motor actuator

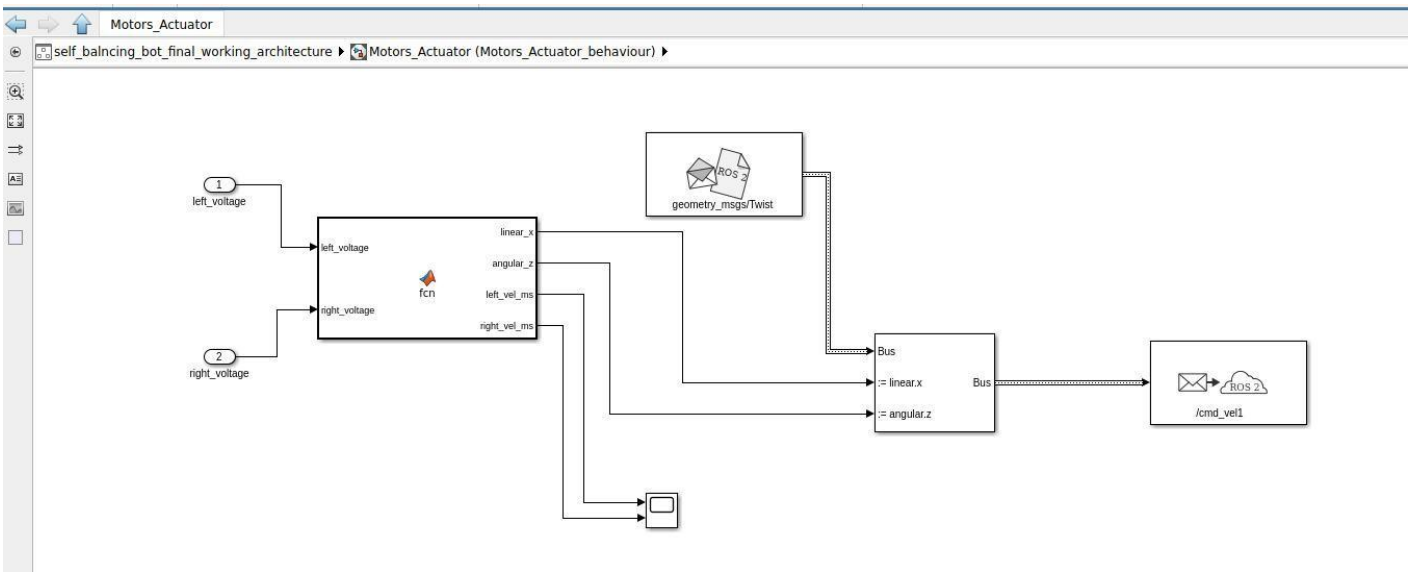


Figure 24: Motors Actuator Subsystem

5.8.1 Motors Actuator Subsystem (ROS 2 Interface)

The final subsystem in the control architecture is the Motors Actuator block, which functions as the output interface between the control system and the simulation environment. While the upstream subsystems operate entirely in the mathematical domain, this block is responsible for protocol translation and message formatting, enabling the computed control commands to be transmitted to Ignition Gazebo through the ROS 2 middleware.

5.8.2 Input Signals

The subsystem receives the final, safety-clamped voltage commands from the Motor Drivers subsystem:

- left_voltage: Simulated voltage command for the left motor *(Range: −12.0 V to +12.0 V)*
- right_voltage: Simulated voltage command for the right motor *(Range: −12.0 V to +12.0 V)*

These signals represent the final control effort approved for actuation and are guaranteed to remain within realistic hardware limits.

5.8.3 Kinematic Conversion (MATLAB Function Block)

A key challenge in the integration process arises from a data format mismatch. The control algorithm computes individual wheel actuation values (V_{Left}, V_{Right}), whereas the Gazebo Differential Drive plugin expects a high-level motion command in the form of a Twist message, consisting of:

- Linear velocity v(forward motion)

- Angular velocity ω(yaw rate)

To bridge this gap, the MATLAB Function block (fcn) implements inverse differential drive kinematics, converting the wheel voltages into an equivalent motion command.

Linear Velocity:

$$v = \frac{V_{Left} + V_{Right}}{2}$$

Angular Velocity:

$$\omega = \frac{V_{Right} - V_{Left}}{L}$$

where Ldenotes the effective wheelbase of the robot.

This transformation abstracts the low-level motor voltages into a high-level kinematic command, effectively instructing Gazebo to move the robot at the specified speed and turning rate. The defined maximum voltage of ±5 V corresponds to the maximum allowable simulated velocity.

5.8.4 Message Construction (Bus Assignment)

ROS 2 messages are structured bus objects, requiring explicit field assignment. The following steps are performed:

- Blank Message Source:
  The geometry_msgs/Twist block generates an empty message template.
- Bus Assignment:
  The calculated kinematic values are inserted into the message fields:
  - Linear velocity v→ Linear.x
  - Angular velocity ω→ Angular.z

All other message fields remain zero, ensuring clean and deterministic communication.

5.8.5 Data Transmission (ROS 2 Publish)

The completed message is transmitted using the ROS 2 Publish block.

- Topic: /cmd_vel1 (bridged internally to /cmd_vel)
- Action: The message is broadcast onto the ROS 2 network

The ROS–Ignition Bridge immediately converts this ROS 2 message into Ignition Transport format and forwards it to the Gazebo Differential Drive plugin, which applies the corresponding forces to the wheel joints. This process completes the closed-loop control cycle by translating controller output into physical motion

Figure 25: Balancing voltage is converted to balancing velocity

5.8.6 Actuator Response Analysis

To validate the performance and safety of the actuation subsystem, the motor voltage outputs were monitored using a Simulink Scope during an active balancing experiment.

Analysis of the Control Signal

As illustrated in Figure 5.8, the graph presents the real-time voltage commands applied to the left and right motors while the robot maintains upright balance.

High-Frequency Corrective Action

The control signal exhibits rapid, high-frequency oscillations, frequently switching between positive and negative voltage values. This behaviour is characteristic of a high-gain PD controller regulating an inherently unstable inverted pendulum system.

These rapid sign changes correspond to continuous micro-corrections applied by the motors. Each adjustment counteracts small deviations in tilt caused by gravity, sensor noise, or dynamic coupling effects. In effect, the motors are constantly generating corrective torque on a millisecond time scale, which manifests as a controlled "vibration" that maintains upright stability.

Saturation Verification

The voltage waveform clearly demonstrates clipping at the defined limits of $+5.0$ and -5.0. This confirms that the saturation logic implemented in the Motor Driver subsystem is operating as intended.

Even when the PID controller attempts to command large corrective efforts during transient

disturbances, the actuator output is constrained within safe and physically realistic bounds. This behavior accurately emulates the voltage ceiling imposed by real motor drivers and battery systems.

Conclusion

The actuator response confirms correct integration between the control algorithm and the actuation interface. The motors respond aggressively enough to maintain balance while respecting hardware-imposed limits, ensuring both stability and safety. This validation demonstrates that the actuation subsystem is well-suited for sustained balancing and dynamic maneuvering experiments.

CHAPTER 6

Testing and Validation

6.1 Unit Testing: IMU Data Verification

To ensure the integrity of the control loop, the IMU data transmission pipeline was isolated and tested before enabling the motors. The objective was to confirm that the real-time orientation data from Gazebo was accurately received and decoded by the MATLAB controller without corruption or delay.

Test Procedure:

Source: The robot was spawned in Gazebo and allowed to tilt freely.

Verification: The ros2 topic echo /imu command was executed in a terminal to verify the raw data stream.

Validation: Simultaneously, the decoded current angle signal was monitored in a MATLAB Simulink Scope.

Observation: The data streams were perfectly synchronized. As the robot tilted in the simulation, the Quaternion values in the ROS terminal updated instantly, and the MATLAB scope plotted a corresponding smooth curve. This validated that the ROS-Ignition Bridge and the Quaternion-to-Euler Block were functioning correctly with negligible latency.

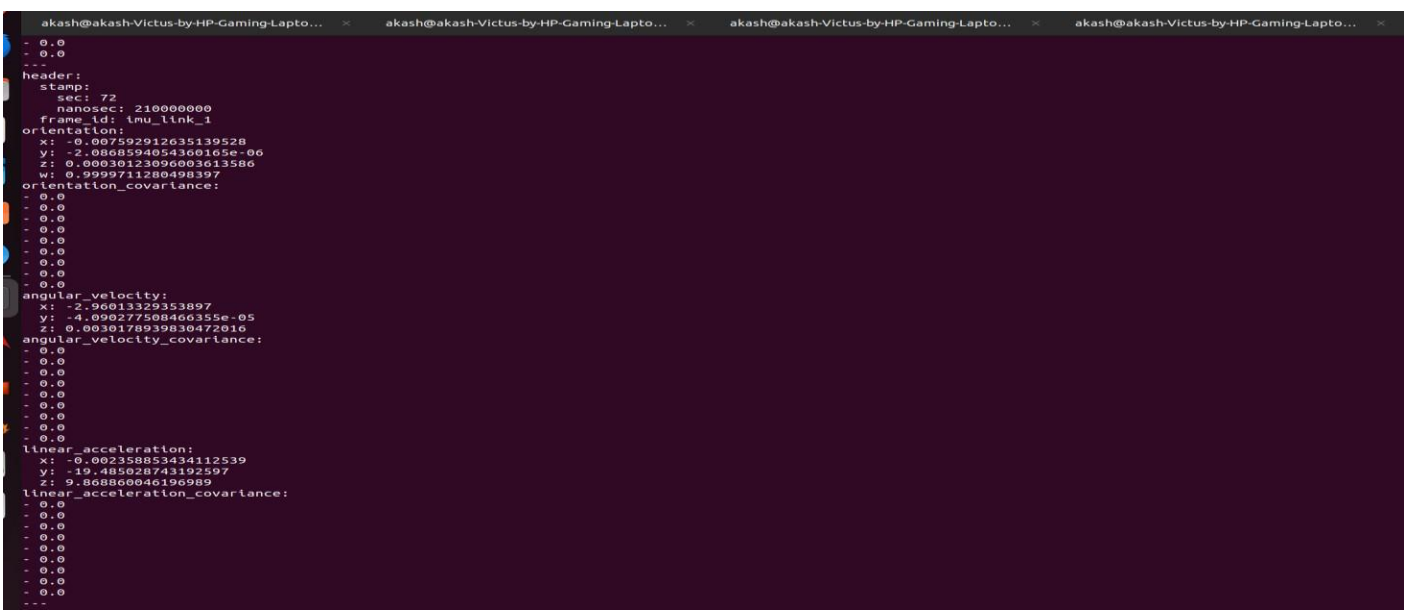


Figure 26: RAW IMU data from Ignition Gazebo

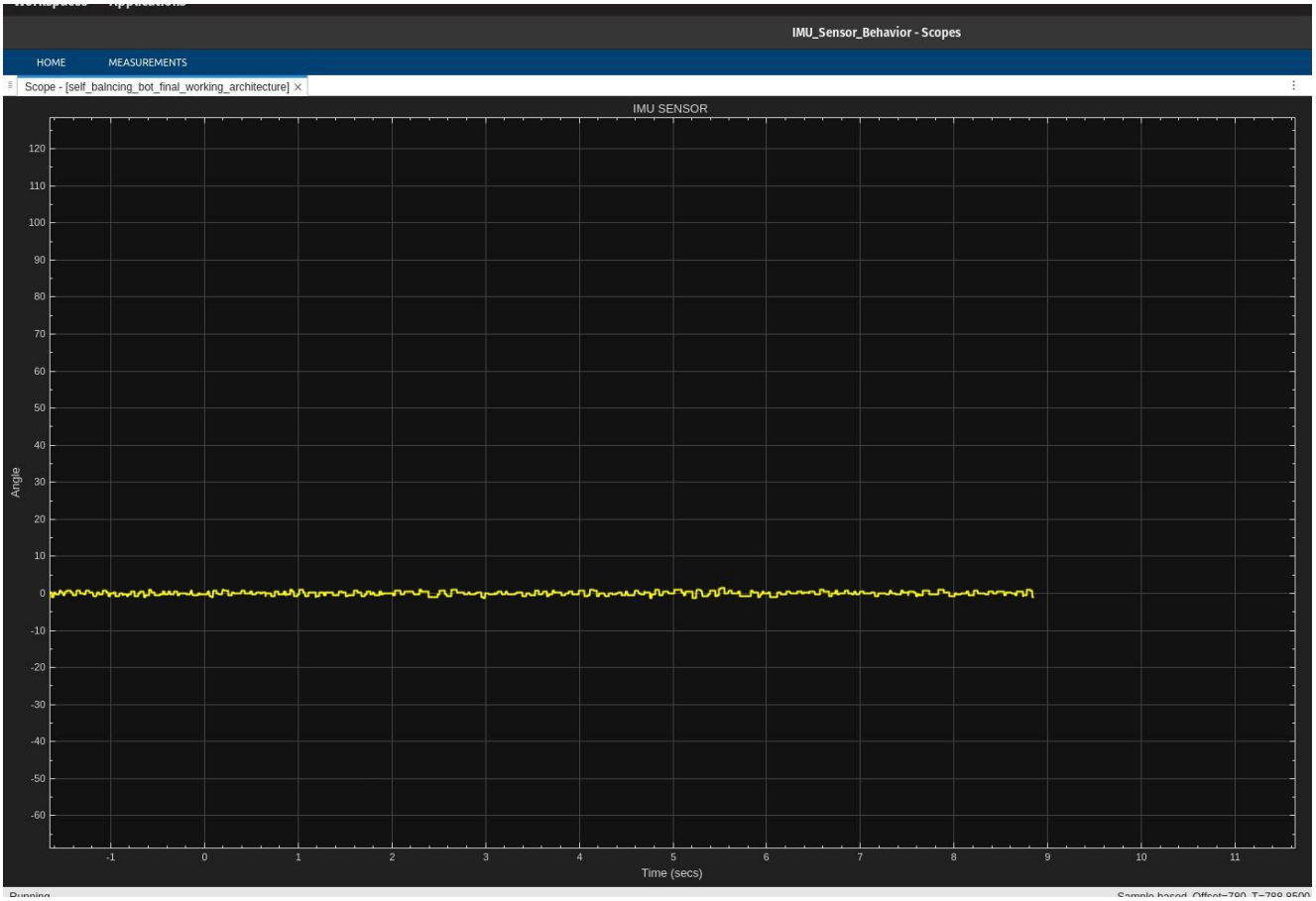


Figure 27: IMU data same as raw data published by Ignition Gazebo

6.2 Addressing Frequency Mismatch (The Staircase Effect)

During early closed-loop testing, the control system exhibited erratic motor twitching, even in the absence of sensor noise or external disturbances. A detailed analysis of the simulation timing revealed that the issue originated from a sampling frequency mismatch between MATLAB/Simulink and Ignition Gazebo.

The Issue: Staircase Phenomenon

MATLAB was executing its control loop at a higher rate than Gazebo was publishing IMU updates. As a result, the controller repeatedly sampled the same sensor value multiple times before a new update was received.

- Visual Effect:
  The pitch angle signal appeared as a *staircase waveform*—remaining constant for several milliseconds and then abruptly jumping to a new value.
- Mathematical Consequence:
  This behavior severely corrupted the Derivative (D) term of the PID controller. Since

$$D = \frac{\Delta e(t)}{\Delta t},$$

the derivative evaluated to zero during the flat steps (no change in error) and to an extremely large value during the sudden jumps.
This caused the controller to alternately command zero corrective effort and maximum corrective effort, leading to violent motor jitter and unstable actuation.

The Solution: Fixed-Step Synchronization
To eliminate this artifact, the Simulink solver was reconfigured as follows:

- Solver Mode: Fixed-Step
- Step Size: 0.01 s (100 Hz)

This configuration forced MATLAB to execute exactly one control update per Gazebo physics update, ensuring temporal alignment between sensor sampling and controller execution.

6.3 PID Tuning Results

After verifying the integrity of the sensor data pipeline and synchronizing the solver execution rates, the PID controller was activated in closed-loop mode. The controller gains were tuned iteratively to achieve an optimal balance between fast corrective response and stable operation. The final tuned values were:
$K_p = 18.0, K_d = 2.5$

The integral term was omitted to avoid wind-up effects in the inherently unstable inverted pendulum system.
ANGLE VS TIME GRAPH (STABLE AT 0 DEGREES)]
Analysis
The angle response shown in Figure 6.2 demonstrates stable closed-loop behavior. The robot's tilt angle is effectively regulated around the desired setpoint of 0°. Only small, bounded oscillations within approximately $\pm 1^\circ$are observed.
These oscillations indicate that the controller is continuously applying micro-corrections to counteract gravity, sensor noise, and numerical disturbances. Importantly, no sustained oscillations or divergence are present, confirming that the proportional gain provides sufficient corrective authority while the derivative gain effectively damps the system response.

6.3.1 Response to External Disturbances
To evaluate the robustness of the controller, the balanced robot was subjected to manual disturbances in the simulation, representing sudden external pushes that forced the system away from equilibrium.

Analysis
The control effort plot illustrates the characteristic "React-and-Recover" behavior of the Switching PD controller:

- Disturbance Response:
  Sharp, high-amplitude spikes appear in the PWM signal at the moments when external forces are applied. These spikes indicate that the controller instantly detects the rapid change in tilt angle.

- Saturation Behavior:
  The controller immediately drives the motor command to the defined safety limits of ± 250PWM, producing maximum available counter-torque to arrest the fall.
- Recovery Phase:
  Once stability is restored, the PWM signal rapidly returns to a low-amplitude steady state. This confirms that the controller does not remain saturated and avoids excessive oscillations after recovery.

The results validate the controller's ability to rapidly reject disturbances, safely utilize motor authority, and re-establish equilibrium without instability.

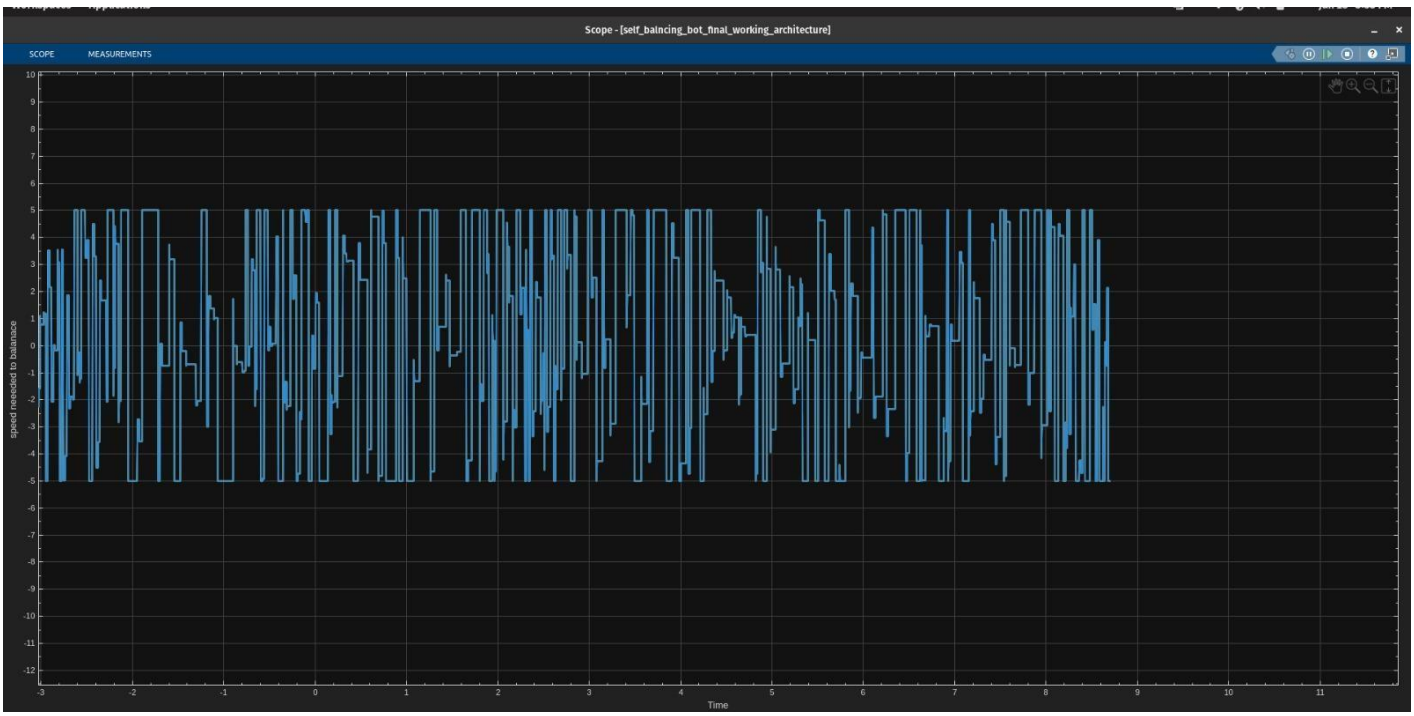


Figure 28: Motor Actuation

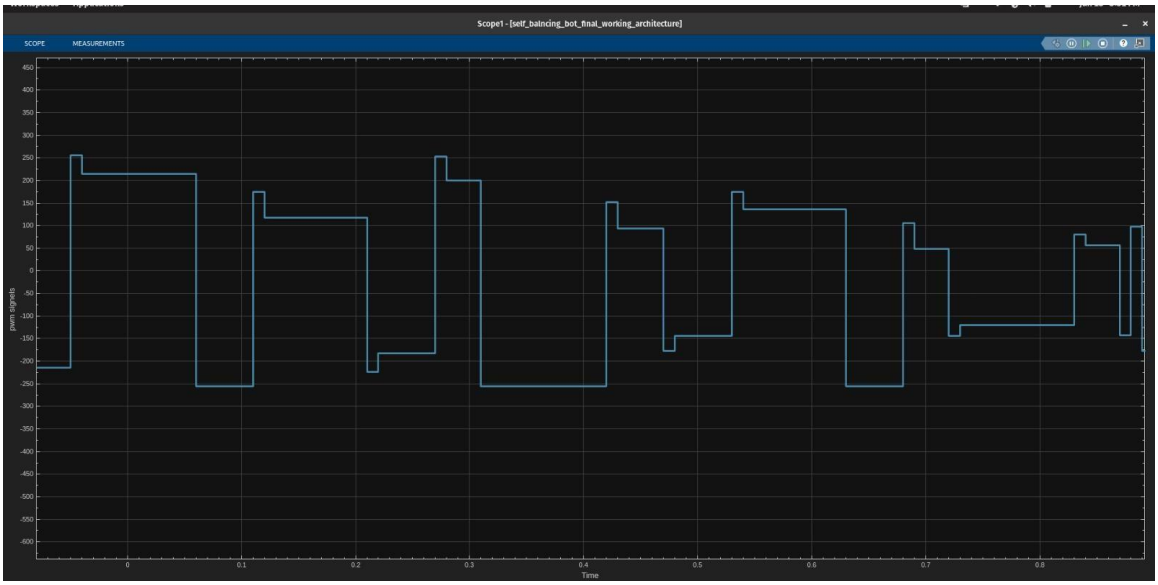


Figure 29: PWM signal

6.4 Robustness Testing Setup: Static Payload Integration

To evaluate the robustness of the proposed control system against physical model uncertainties, a dedicated stress-testing scenario was designed involving a deliberate modification of the robot's inertial properties. This experiment aimed to assess the controller's ability to maintain stability under conditions that deviate from the nominal system model used during controller tuning.

Test Configuration

A static cargo block was rigidly attached to the robot chassis using a fixed joint. The attachment intentionally altered both the centre of mass (CoM) and the moment of inertia of the system, thereby forcing the PID controller to operate on a physical model different from the one for which it was originally designed and tuned. This configuration simulates real-world operational scenarios such as carrying loads or equipment mounted above the chassis.

Payload Specifications

- Geometry: Rectangular prism
- Dimensions: $0.3\ \mathrm{m} \times 0.2\ \mathrm{m} \times 0.1\ \mathrm{m}$
- Mass: $0.5\ \mathrm{kg}$
- Mounting Position: Rigidly fixed at a vertical offset of

$z = 0.07\ \mathrm{m}$

from the base link origin, placing the payload directly above the IMU sensor

- Material Visualization: Rendered in warning red to clearly distinguish the payload from the robot chassis during simulation and analysis

Objective

The primary objective of this robustness test was to verify whether the PID gains

$K_p = 18.0, K_d = 2.5$

—originally derived for the unloaded robot—provided sufficient stability margins to balance

the system under a top-heavy configuration without requiring controller re-tuning. Successful stabilization under this altered inertial condition would demonstrate the robustness and generalization capability of the control strategy

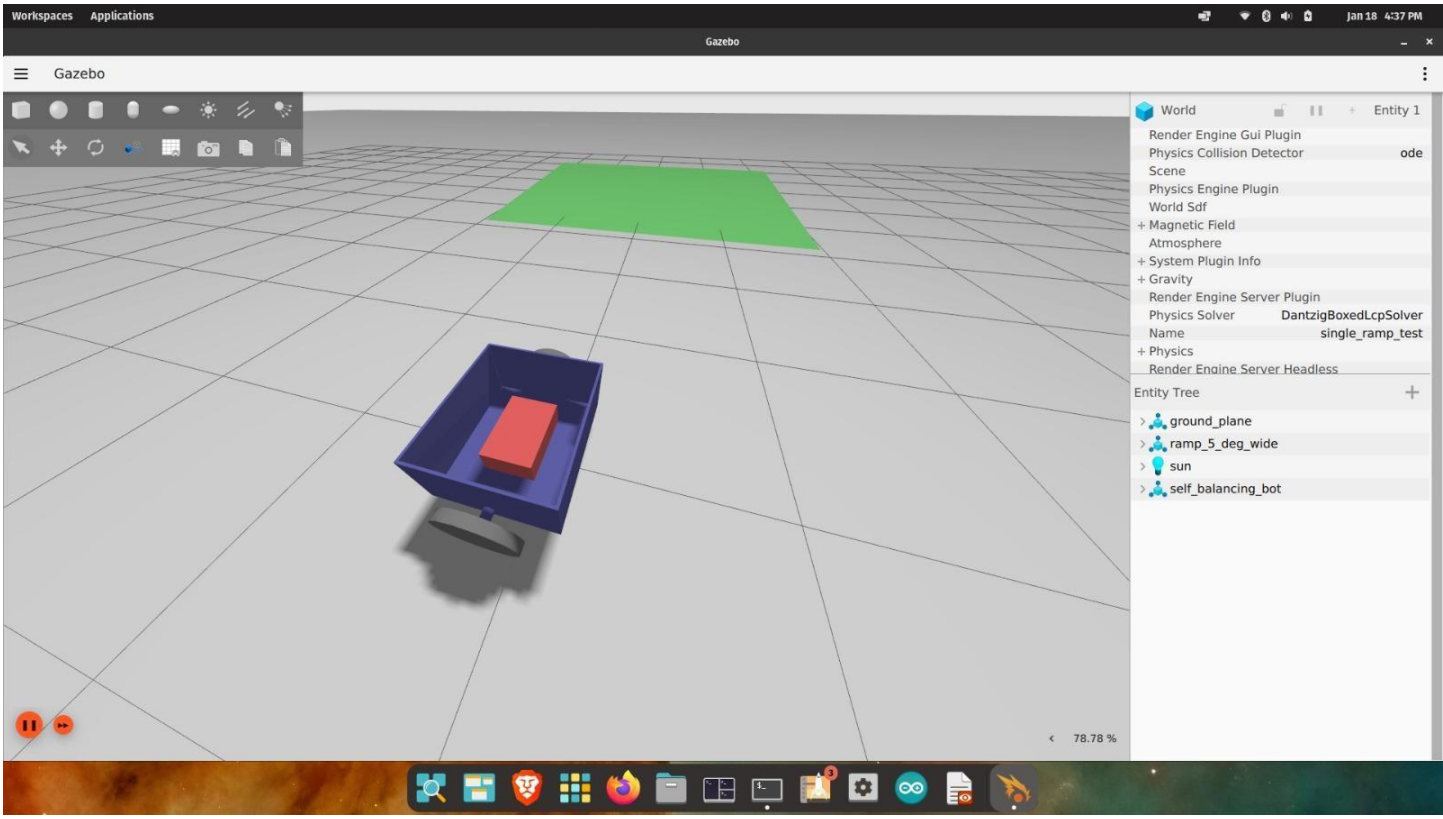


Figure 30: Bot with Payload

CHAPTER 7

Results and Discussion

7.1 Balancing Performance (Stationary)

The primary objective of this project was to stabilize an inherently unstable inverted pendulum system. The effectiveness of the control architecture is most clearly demonstrated by comparing the robot's behavior in open-loop and closed-loop configurations.

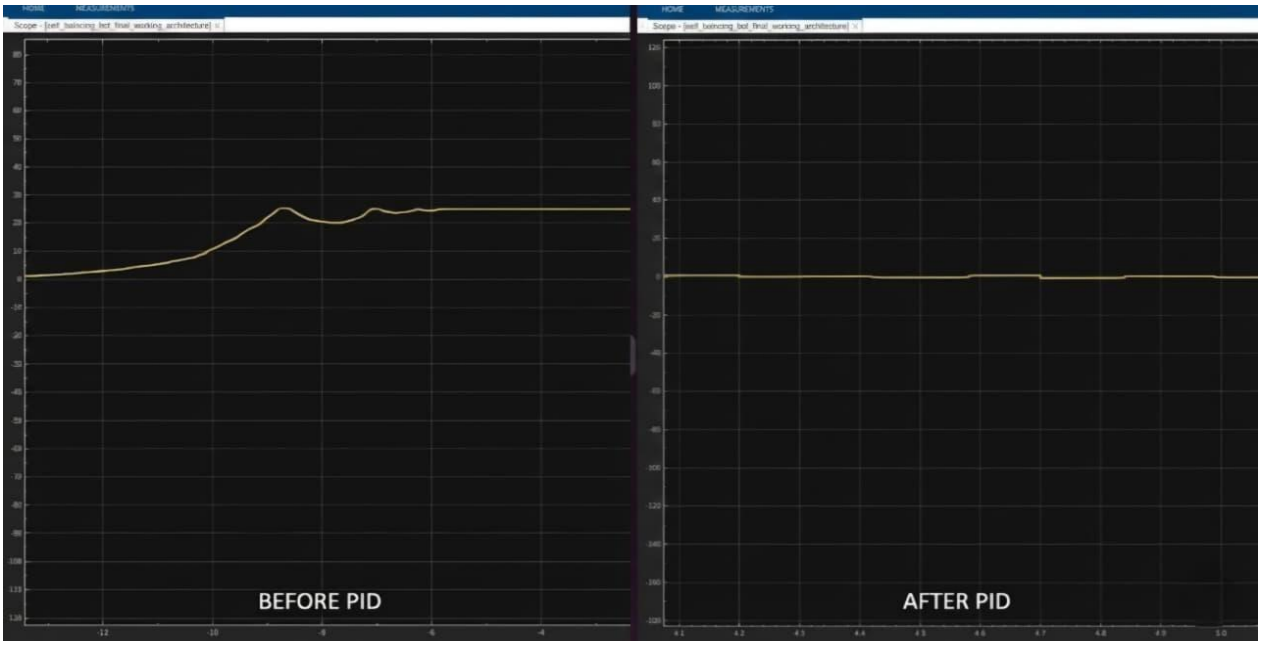


Figure 31: Difference in imu data before and after Pid turned ON

Case 1: Open-Loop Response (Control System OFF)

In the initial test, the robot was spawned in the Ignition Gazebo environment with the PID controller fully deactivated:

$K_p = 0, K_d = 0$

Observation

Immediately upon simulation start, the robot succumbed to gravitational instability. Within approximately 0.5 seconds, the pitch angle diverged beyond the critical recovery threshold ($> 15°$).

Result

The chassis collapsed onto the ground plane without any corrective response. This behavior confirms that the mechanical structure is statically unstable and possesses no passive self-righting capability.

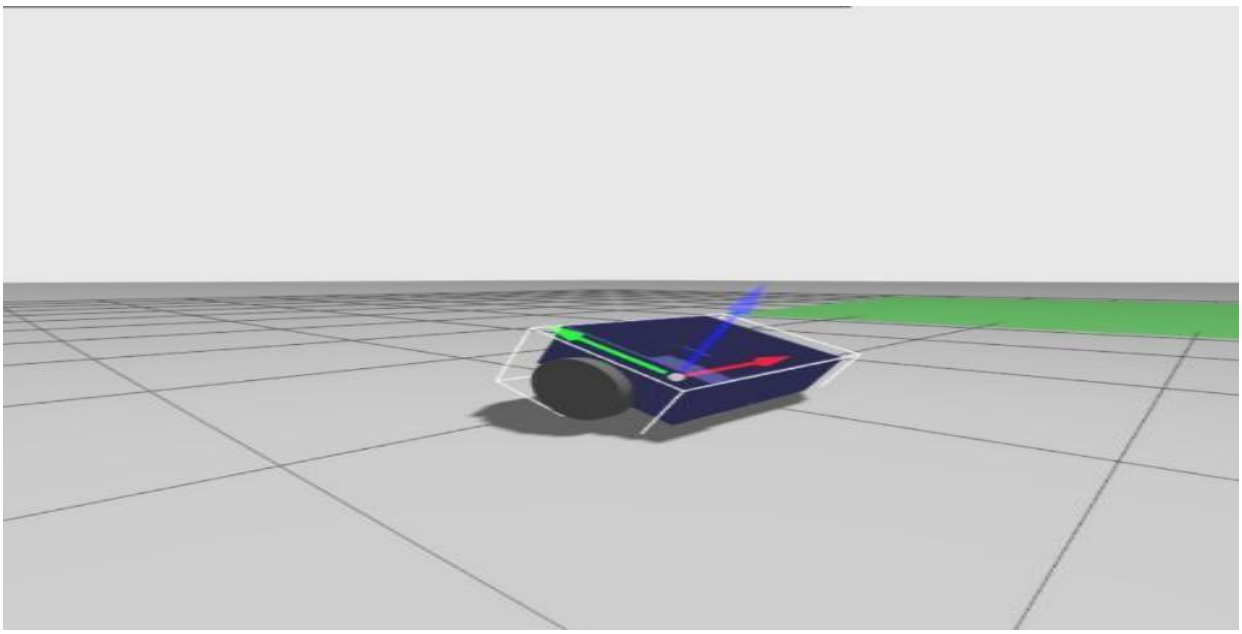


Figure 32: Bot collapse on to ground without PID

Case 2: Closed-Loop Response (Control System ON)

The simulation was repeated with the tuned PID controller activated:

$K_p = 18.0, K_d = 2.5$

Observation

The robot was initialized with a slight tilt of approximately $5°$. Upon simulation start, the motors engaged instantly, generating corrective torque in the direction of the fall.

Result

The robot rapidly recovered to the vertical setpoint ($0°$) and maintained a stable upright posture indefinitely. Small, continuous wheel motions—observed as *micro-shuddering*—

indicated ongoing balance correction. Despite the system's inherent instability, the controller effectively counteracted gravity at all times.

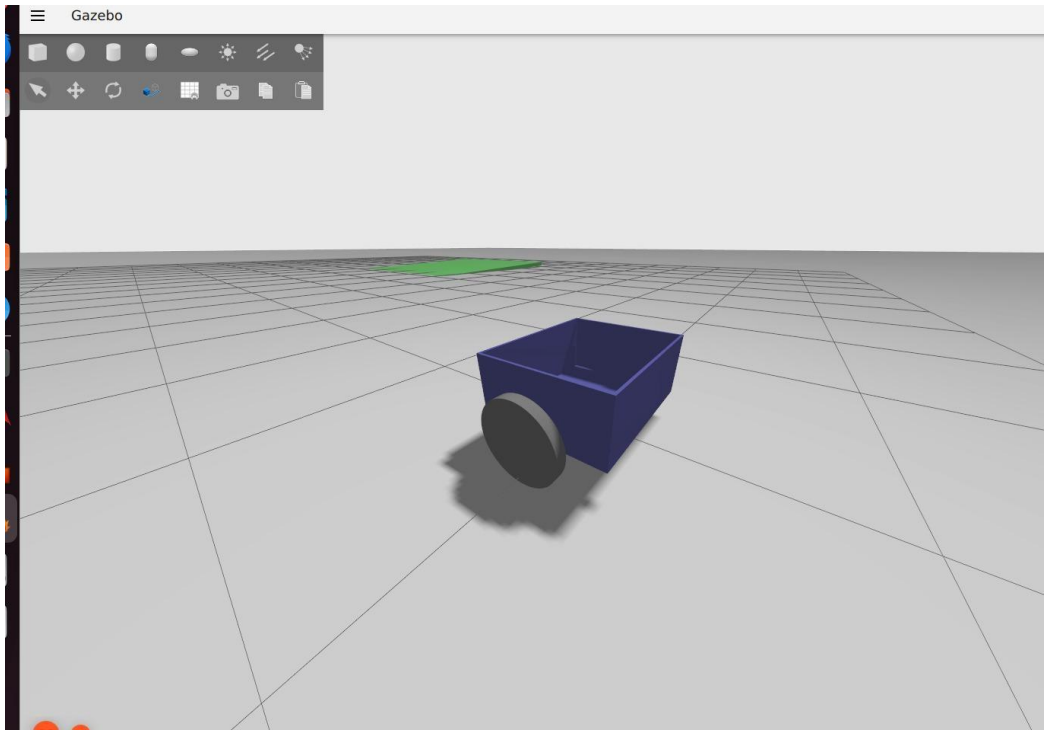


Figure 33: Bot self-balancing after PID turned is ON

7.2 Maneuverability Analysis (WASD Control)

Unlike conventional wheeled robots, a self-balancing robot cannot generate linear motion without intentionally disturbing its equilibrium. Forward and backward motion is achieved by shifting the balance point rather than directly commanding wheel velocity. The maneuverability logic was evaluated using keyboard teleoperation (WASD inputs).

Forward and Backward Motion ('W' and 'S' Commands)

When a forward command ('W') was issued via the ROS 2 terminal, the robot did not immediately accelerate. Instead, motion occurred through a three-phase physical process:

Phase 1: Target Lean Adjustment

The maneuverability subsystem shifted the target pitch angle to $-3°$, intentionally creating a virtual imbalance.

Phase 2: Controller Reaction

The PID controller detected that the robot, currently at $0°$, was “too upright” relative to the new target. The wheels briefly moved backward, causing the chassis to lean forward.

Phase 3: Steady-State Motion

Once the robot achieved the $-3°$tilt, the centre of mass shifted forward. To prevent falling, the controller continuously drove the wheels forward, resulting in sustained linear motion.

Backward motion (‘S’) exhibited symmetric behaviour with an opposite tilt direction.

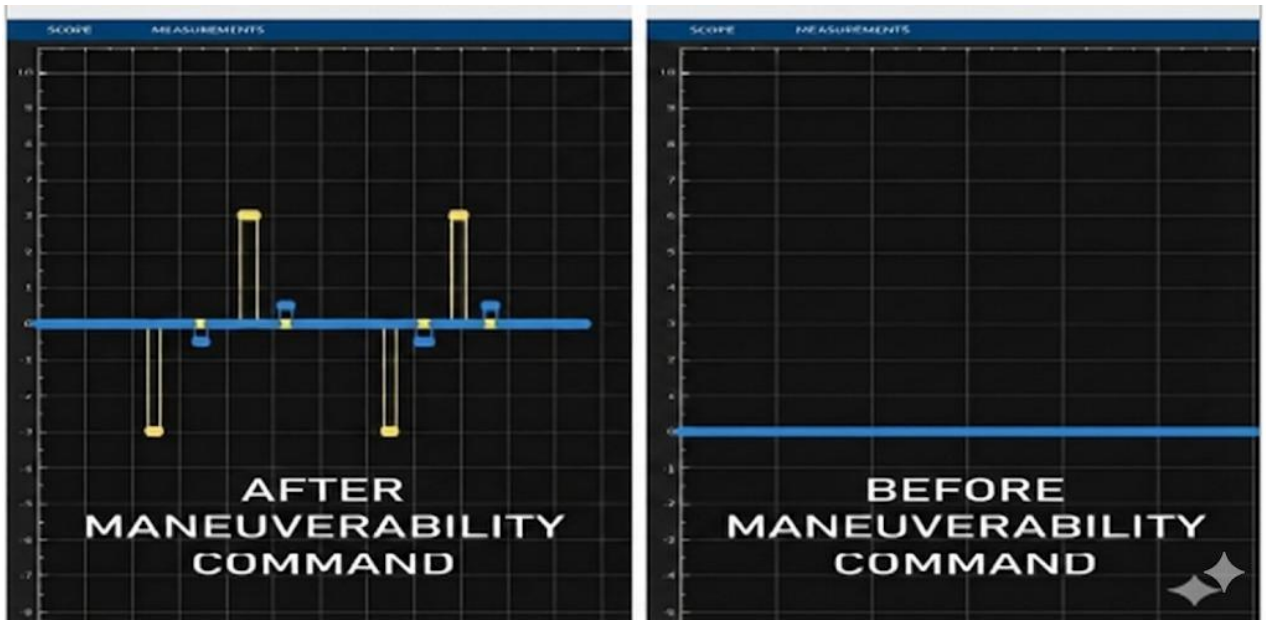


Figure 34: Before and After input from user command

7.3 Stability Analysis during Turning

Turning introduces significant dynamic risk for a two-wheeled balancing robot, as differential wheel speeds generate centrifugal forces that can destabilize the inverted pendulum. The effectiveness of the Differential Drive Mixing and Stability Manager logic was evaluated under both stationary and dynamic conditions.

Turning Performance (‘A’ and ‘D’ Commands)

Stationary Turning

With the robot balanced at $0°$, issuing a turn command caused the wheels to spin in opposite directions (Left: positive, right: negative). The robot executed a zero-radius turn (pirouette) while maintaining vertical stability.

Turning While in Motion

When turning commands were applied during forward motion, the turning bias was smoothly combined with the balancing command. The robot followed smooth arc trajectories without oscillation or loss of balance.

Safety Verification: Stability Manager

During aggressive maneuvers and abrupt stops, the robot's tilt occasionally exceeded the predefined safe threshold ($\pm 3°$). In these situations:

- The controller automatically disabled turning inputs
- Full motor authority was redirected toward balance recovery
- Turning commands were reinstated only after stability was regained

This behavior confirms the correct operation of the Rescue Mode logic, ensuring that stability is always prioritized over maneuverability.

7.4 Payload Stability Analysis

The introduction of a 0.5 kg static payload represented a substantial percentage increase in the robot's total mass and caused a deliberate upward shift in the system's center of mass (CoM). The dynamic response of the robot under this loaded condition was analyzed to assess the robustness of the developed control strategy against variations in physical parameters.

Performance Observation

Upon initialization of the simulation with the attached payload, the robot successfully achieved and maintained an upright equilibrium. No instability or divergent behavior was observed during the startup phase, indicating that the controller was capable of handling the altered inertial characteristics of the system.

Transient Response

Compared to the unloaded configuration, the system exhibited a slightly slower transient response when subjected to initial disturbances. This behavior is consistent with the increased rotational inertia (I_{yy}) introduced by the elevated payload, which requires higher corrective torque to achieve the same angular acceleration. Despite this increase in inertia, the controller response remained smooth, without excessive oscillations or overshoot.

Steady-State Stability

In steady-state operation, the robot maintained a stable vertical orientation despite the top-heavy configuration. The existing proportional gain

$K_p = 18.0$

generated sufficient corrective torque to counteract gravitational moments induced by the raised CoM. Angular deviations remained well within the safe operational limit of

$\pm 2°$

demonstrating effective balance control without sustained oscillations or drift.

Conclusion on Robustness

This payload stability test confirms that the developed Switching PD Controller possesses a substantial stability margin. The controller successfully stabilized a system with significantly altered physical parameters—specifically increased mass and moment of inertia—without requiring gain re-tuning, gain scheduling, or adaptive control mechanisms. Such robustness is essential for real-world deployment, where payload mass and distribution may vary dynamically during operation.

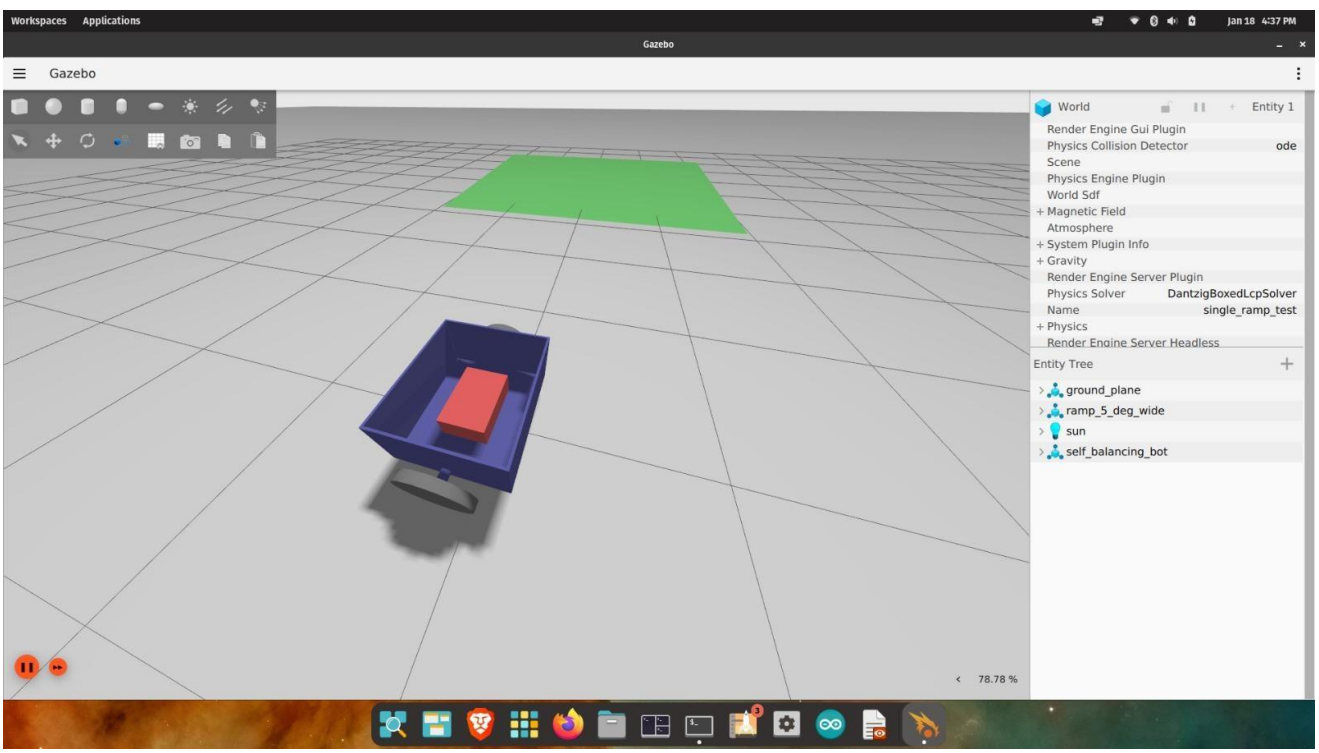


Figure 35: Bot self-balancing with payload placed on it

7.5 Terrain Travers ability: Ramp Climbing Analysis

In addition to evaluating flat-ground maneuverability and payload robustness, the robot's ability to negotiate inclined terrain was assessed using a standard ramp model within the Gazebo simulation environment. This experiment aimed to analyze the limitations of the self-balancing control strategy under gravitational loading and to identify feasible operational strategies for slope traversal.

Stationary Ascent Test (Failure Case)

Initial trials were conducted with the robot positioned at rest directly at the base of the ramp. Under this configuration, the robot was unable to initiate upward motion and failed to climb the incline.

Physics of Failure

For a self-balancing robot to translate forward, it must deliberately lean in the direction of motion, effectively allowing controlled "falling" to generate propulsion. On an inclined surface, the robot is already tilted backward relative to the slope due to gravitational effects. To generate sufficient forward motion, the controller must command an excessive forward lean to counteract the downhill component of gravity.

This requirement caused the motor drivers to saturate, reaching maximum PWM output almost instantaneously. As a result, the actuators were unable to generate the additional torque required to overcome the combined effects of gravity and inertia, preventing the robot from initiating the climb from a stationary condition.

Momentum-Based Ascent (Success Case)

To overcome the limitations observed in the stationary ascent test, a run-up strategy was implemented, leveraging the system's linear momentum.

Procedure

The robot was positioned approximately 2 meters away from the base of the ramp on level ground. It was then accelerated to near-maximum forward velocity using the 'W' command prior to contacting the incline.

Result

By utilizing accumulated linear momentum (kinetic energy), the robot successfully transitioned from the flat surface onto the inclined plane. The initial forward velocity reduced the instantaneous torque demand on the motors, enabling the robot to climb the ramp and reach the top without motor saturation. This dynamic ascent approach effectively compensated for the static torque limitations observed during the stationary test.

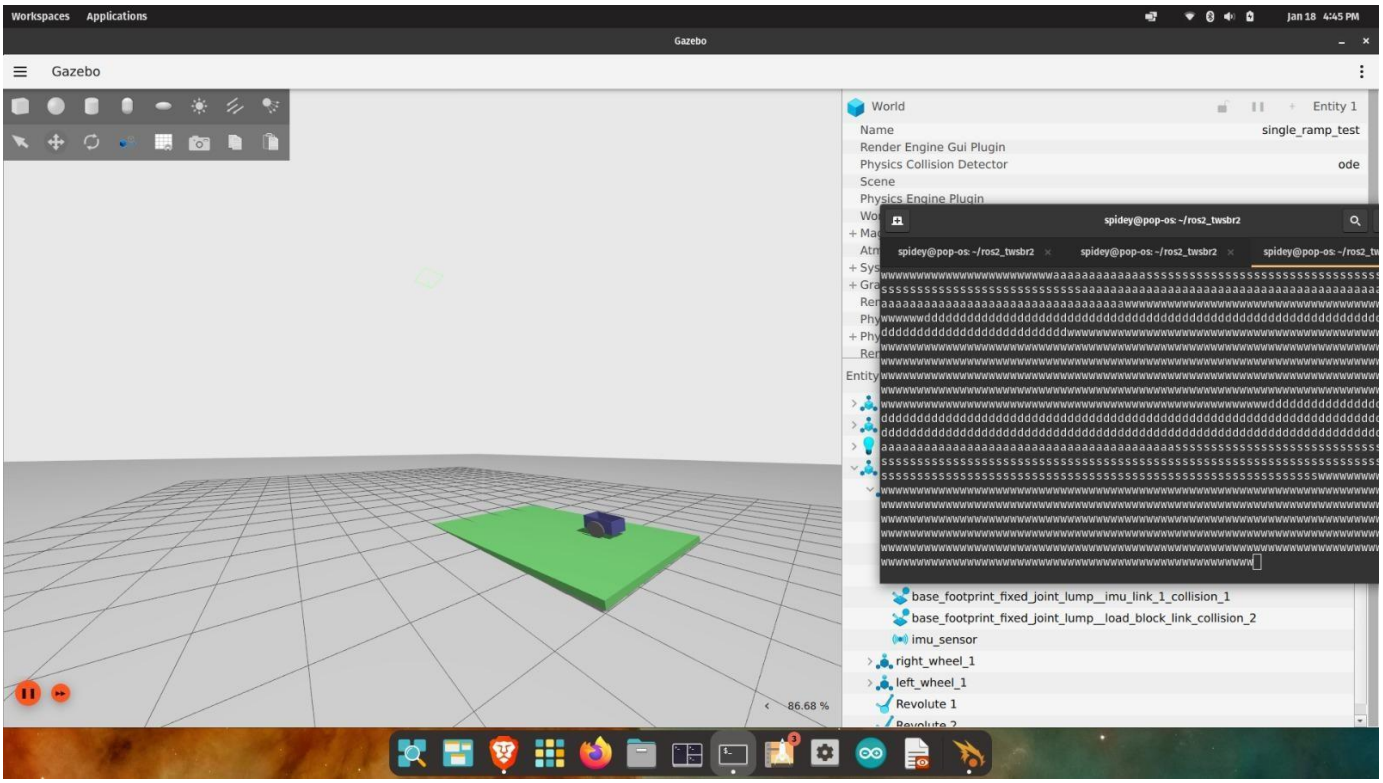


Figure 36: Bot climbing on ramp

.

CHAPTER 8

Applications, Limitations, and Future Scope

8.1 Applications

The design philosophy of the two-wheeled self-balancing robot (TWIP) extends beyond a purely academic demonstration of control theory. By leveraging the inverted pendulum principle, the platform offers distinct mechanical and operational advantages over statically stable robots (such as four-wheeled rovers), making it suitable for several real-world deployment scenarios.

8.1.1 Last-Mile Intralogistics and Warehousing

The robot's differential-drive architecture enables rotation about its vertical axis with a **zero** turning radius. This capability allows efficient navigation within narrow warehouse aisles, storage racks, and congested indoor environments where conventional Ackermann-steered carts struggle to maneuver.

The custom chassis design, featuring a lowered center of mass and a flat upper payload deck, is optimized for transporting small-to-medium parcels in last-mile intralogistics applications within factories, warehouses, and distribution centers.

8.1.2 Dynamic Payload Transport (Active Suspension Effect)

Unlike rigid wheeled platforms, the self-balancing mechanism inherently behaves as a dynamic suspension system. During acceleration or deceleration, the robot tilts to counteract inertial forces instead of transmitting them directly to the payload.

Benefit:

This lean-into-motion behavior significantly reduces the effective g-forces experienced by the payload. As a result, the robot is well suited for transporting fragile goods (such as electronics or glassware) and liquid containers (such as medical or chemical samples), where minimizing vibration, shock, and sloshing is critical.

8.1.3 Indoor Service and Human–Robot Interaction

With a compact footprint approximately equal to the wheel width, the robot occupies minimal ground space, making it ideal for crowded indoor environments such as hospitals, offices, laboratories, and airports. It can navigate through pedestrian traffic while delivering documents, tools, or medical supplies without obstructing human movement.

Furthermore, the robot's ability to actively recover from external disturbances through re-balancing enhances operational safety in shared human environments, where rigid platforms may stall or tip over after minor impacts.

8.1.4 Mobile Surveillance and Inspection

The upright configuration of the TWIP naturally provides an elevated mounting position for perception sensors. A camera or sensor suite mounted near the top of the chassis benefits from a wider field of view compared to ground-level rover platforms.

Additionally, the ability to perform 360-degree rotation in place enables full environmental scanning using a static camera or LIDAR sensor, eliminating the need for mechanical pan– tilt units and simplifying the sensing architecture.

8.2 Limitations

Although the simulation results validated the effectiveness of the proposed control architecture, several limitations were identified during robustness and stress testing. These constraints primarily arise from the use of a linear PID-based controller applied to an inherently non-linear system.

8.2.1 Incline travers ability (Torque Saturation)

The robot experiences difficulty when attempting to climb inclined surfaces from a stationary position.

Physical Explanation:

On a slope, the gravitational force vector introduces a backward pitch bias relative to the robot body frame. To ascend the incline, the robot must lean forward aggressively. However, the PID controller continuously attempts to regulate the pitch angle toward zero, effectively opposing the required forward lean.

Observed Behavior:

This conflict causes the motor commands to rapidly reach saturation (maximum PWM), preventing the generation of sufficient torque to initiate upward motion. As a result, the robot can currently climb ramps only by employing a momentum-based approach, where kinetic energy acquired on flat terrain assists the ascent.

8.2.2 Sensitivity to Payload Variations

The implemented PID gains are tuned for the robot's nominal mass distribution.

Issue:

Introducing payloads alters the system's centre of mass and moment of inertia, parameters that are not explicitly accounted for by the controller.

Consequence:

While the controller demonstrated robustness during the 0.5 kg static payload test, significantly heavier, uneven, or dynamically shifting loads may exceed the available stability margin, leading to slower recovery, increased oscillations, or loss of balance.

8.2.3 Communication Latency in the Control Loop

The simulation architecture relies on communication between Ignition Gazebo (physics simulation) and MATLAB Simulink (control computation) via the ROS 2 middleware.

This distributed setup introduces non-zero communication latency and jitter. In near-fall conditions where corrective action must occur within a few milliseconds, delayed sensor feedback can degrade the effectiveness of the derivative control action, potentially compromising stability.

8.3 Future Scope

To advance the system from a validated simulation prototype toward a deployment-ready robotic platform, the following enhancements are proposed:

8.3.1 Dynamic Payload Simulation (Loose Cargo)

Current robustness testing employed a rigidly attached payload using a fixed joint. Future work will focus on dynamic payload simulation, where the cargo is modelled as a separate physical entity placed freely on the payload deck.

Objective:

This approach will evaluate the robot's ability to handle load shifting, frictional effects, and transient disturbances caused by loose cargo. It will necessitate the development of soft-start and acceleration-limiting strategies to prevent cargo slippage.

8.3.2 Adaptive and Robust Control Strategies

To address payload sensitivity and non-linear dynamics, the PID controller can be replaced or augmented with advanced control techniques such as Linear Quadratic Regulation (LQR) or Model Predictive Control (MPC).

These controllers operate on a full state-space representation of the system and can adapt motor torque commands based on estimated changes in mass, inertia, and operating conditions, thereby improving robustness and performance.

8.3.3 Incline-Aware "Climb Mode"

An incline detection mechanism can be developed using sensor fusion between accelerometer data and wheel encoder feedback.

Implementation Concept:

Upon detecting a slope, the controller would transition into a dedicated climb mode, temporarily relaxing pitch constraints and allowing higher torque output. This would enable the robot to climb ramps from a stationary position without relying on a momentum-based approach.

8.3.4 Autonomous Navigation and SLAM

The current system operates under teleoperation. Future development will integrate a 2D LIDAR or depth camera to enable Simultaneous Localization and Mapping (SLAM).

This enhancement would allow fully autonomous navigation, obstacle avoidance, and goal-directed motion, enabling the robot to operate independently in structured and semi- structured indoor environments.

CHAPTER 9

Conclusion

The primary objective of this project was to design, model, and control a Two-Wheeled Self-Balancing Robot (TWSBR) using a modern Model-Based Design (MBD) workflow. By integrating Autodesk Fusion 360, ROS 2 Humble, Ignition Gazebo, and MATLAB/Simulink, the project successfully demonstrated a complete *Sim-to-Real–ready* development pipeline that bridges mechanical design, dynamic modelling, and control system implementation.

The development process began with the conceptualization of a statically unstable inverted pendulum system. The mechanical structure was meticulously designed in Fusion 360 and exported to a URDF format, overcoming multiple compatibility and inertia-definition challenges. This enabled the creation of a physics-accurate simulation model within Ignition Gazebo. A robust bidirectional communication interface was established using ROS 2, allowing real-time exchange of sensor data and actuator commands between the simulation environment and the control framework.

The core control architecture, implemented in MATLAB/Simulink, validated the effectiveness of a Switching PD Controller for stabilizing a highly non-linear system. Through iterative gain tuning, the controller achieved a stable upright equilibrium with pitch angle deviations constrained to within ±2° under nominal operating conditions. Additionally, the implementation of custom maneuverability Logic enabled intuitive "Tilt-to-Move" behavior, translating user velocity commands into differential wheel torque while preserving balance. This confirmed the correctness of the differential-drive mixing and motion-control strategy.

Robustness testing formed a critical component of system validation. The robot demonstrated reliable performance under multiple challenging conditions, including:

1. External Disturbances: Successful recovery from push-induced perturbations through rapid corrective torque generation.
2. Payload Variation: Stable balancing with a 0.5 kg static payload, representing a significant percentage increase in system mass, without requiring controller re-tuning.
3. Terrain travers ability: Effective ramp climbing using a momentum-based ascent strategy, highlighting both the system's limitations and its operational adaptability.

Overall, this project confirms that a hybrid simulation methodology—combining high- fidelity physics engines with industry-standard control design tools—provides a powerful and efficient framework for developing complex mechatronic systems. The validated simulation platform establishes a strong foundation for future advancements, including adaptive control, autonomous navigation, dynamic payload handling, and eventual hardware realization of the self-balancing robot.

REFERENCES

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022. [Online]. Available: https://www.science.org/doi/10.1126/scirobotics.abm6074

[2] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Sendai, Japan, 2004, pp. 2149–2154.

[3] E. Arslan, M. Suveren, and S. T. H. Moghaddam, "ROS Gazebo and MATLAB/Simulink co-simulation for cart-pole system: A framework for design optimization," *International Journal of Service Science, Management, Engineering, and Technology*, vol. 14, no. 1, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10025177

[4] MathWorks, "ROS Toolbox: Generate and deploy ROS 2 nodes from Simulink," *MathWorks Documentation*, 2024. [Online]. Available: https://www.mathworks.com/help/ros/ros2-node-generation-and-deployment.html

[5] R. C. O. Ferreira *et al.*, "ROS-based controller for a two-wheeled self-balancing robot," in *Proceedings of the IEEE Latin American Robotics Symposium (LARS)*, 2023. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10025177

[6] Open Robotics, "Ignition Gazebo: A modern physics simulator for robotics," *Gazebo Sim Documentation*, 2024. [Online]. Available: https://gazebosim.org/docs

[7] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.

[8] J. Y. Kim and T. Y. Kim, "Dynamic modelling and control of a two-wheeled self-balancing robot," *International Journal of Control, Automation and Systems*, vol. 18, no. 11, pp. 2845–2856, 2020.

[9] Autodesk, "Fusion 360 to URDF Exporter," *Autodesk App Store*, 2024. [Online]. Available: https://apps.autodesk.com/FUSION/en/Detail/Index?id=8050520623668832